
UNIT 9 PRIORITY QUEUES

Structure

- 9.0 Introduction
- 9.1 Objectives
- 9.2 The Priority Queue Abstract Data Type
- 9.3 Implementing a Priority Queue
- 9.4 Heaps
- 9.5 Sorting with a Priority Queue
- 9.6 Adaptable Priority Queues
- 9.7 Summary
- 9.8 Solutions/ Answer to Check your Progress
- 9.9 Further Readings

9.0 Introduction

Priority queues are fundamental data structures that extend the concept of regular queues by associating each element with a priority value. Priority queues use priority levels for managing the elements.

Priority queues find extensive applications in operating systems for process scheduling, network routing protocols, graph algorithms like Dijkstra's shortest path, and simulation systems. Priority queues can be implemented using heaps.

This unit explores the theoretical foundations and practical implementations of priority queues, with particular emphasis on heap-based implementations and their applications in sorting algorithms.

9.1 Objectives

After completing this unit, you will be able to:

- Understand the fundamental concepts and operations of priority queues
- Distinguish between different priority queue implementations and their trade-offs
- Implement priority queues using various data structures including heaps
- Master heap data structure properties and operations
- Apply heap-based algorithms for efficient sorting (Heap Sort)
- Implement adaptable priority queues with key modification capabilities
- Analyze time and space complexity of priority queue operations
- Apply priority queues to solve real-world problems in scheduling and optimization

9.2 The Priority Queue Abstract Data Type

A priority queue is an abstract data type that maintains a collection of elements, each associated with a priority value. The key characteristic is that elements are served in order of their priority rather than their insertion order.

Key Properties:

- Each element has an associated priority
- Higher priority elements are served before lower priority elements
- Elements with equal priority may be served in any order (or FIFO order in some implementations)

PRIORITY QUEUE OPERATIONS

The essential operations for a priority queue include:

Primary Operations:

- `insert(item, priority)`: Add an item with given priority
- `extract_max()` or `extract_min()`: Remove and return highest/lowest priority item
- `peek()`: Return highest/lowest priority item without removing it
- `is_empty()`: Check if the priority queue is empty
- `size()`: Return the number of elements

Additional Operations:

- `change_priority(item, new_priority)`: Modify the priority of an existing item
- `remove(item)`: Remove a specific item from the queue

APPLICATIONS OF PRIORITY QUEUES

Priority queues have numerous real-world applications:

Operating Systems:

- Process scheduling based on priority levels
- Memory management for page replacement algorithms

Network Management:

- Packet routing with Quality of Service (QoS) requirements
- Bandwidth allocation for different traffic types

Graph Algorithms:

- Dijkstra's shortest path algorithm
- Prim's minimum spanning tree algorithm
- A* search algorithm

Simulation Systems:

- Event-driven simulations where events have timestamps
- Task scheduling in project management

Data Processing:

- Top-K problems (finding K largest/smallest elements)
- Merge operations in external sorting

9.3 Implementing a Priority Queue

ARRAY-BASED IMPLEMENTATION

The simplest approach uses an unsorted array to store priority-item pairs:

```

class ArrayPriorityQueue:
    """Simple array-based priority queue implementation."""

    def __init__(self):
self._data = []

    def insert(self, item, priority):
    """Insert item with priority. O(1) time complexity."""
self._data.append((priority, item))

    def extract_min(self):
    """Remove and return minimum priority item. O(n) time complexity."""
    if self.is_empty():
        raise ValueError("Priority queue is empty")

# Find minimum priority item
min_index = 0
    for i in range(1, len(self._data)):
        if self._data[i][0] < self._data[min_index][0]:
            min_index = i

# Remove and return the item
    priority, item = self._data.pop(min_index)
    return item

    def peek_min(self):
    """Return minimum priority item without removing. O(n) time
complexity."""
    if self.is_empty():
        raise ValueError("Priority queue is empty")

min_priority = float('inf')
min_item = None
    for priority, item in self._data:
        if priority < min_priority:
            min_priority = priority
            min_item = item
    return min_item

    def is_empty(self):
    """Check if queue is empty. O(1) time complexity."""
    return len(self._data) == 0

    def size(self):
    """Return number of items. O(1) time complexity."""
    return len(self._data)

# Example usage
pq = ArrayPriorityQueue()
pq.insert("Task A", 3)
pq.insert("Task B", 1)
pq.insert("Task C", 2)

print(pq.extract_min()) # Output: Task B (priority 1)

```

```
print(pq.peek_min()) # Output: Task C (priority 2)
```

SORTED LIST IMPLEMENTATION

An alternative approach maintains elements in sorted order:

python

```
class SortedListPriorityQueue:
    """Priority queue using sorted list implementation."""

    def __init__(self):
        self._data = [] # Maintained in sorted order by priority

    def insert(self, item, priority):
        """Insert item maintaining sorted order. O(n) time complexity."""
        # Find correct position using binary search
        import bisect
        bisect.insort(self._data, (priority, item))

    def extract_min(self):
        """Remove and return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")

        priority, item = self._data.pop(0)
        return item

    def peek_min(self):
        """Return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")
        return self._data[0][1]

    def is_empty(self):
        return len(self._data) == 0

    def size(self):
        return len(self._data)
```

LINKED LIST IMPLEMENTATION

Using linked lists provides flexibility for dynamic sizing:

python

```
class Node:
    """Node class for linked list implementation."""
    def __init__(self, item, priority):
        self.item = item
        self.priority = priority
        self.next = None

class LinkedListPriorityQueue:
    """Priority queue using sorted linked list."""

    def __init__(self):
        self._head = None
        self._size = 0

    def insert(self, item, priority):
```

```

        """Insert item maintaining priority order. O(n) time complexity."""
        new_node = Node(item, priority)

        # Insert at beginning if empty or highest priority
        if not self._head or priority < self._head.priority:
            new_node.next = self._head
            self._head = new_node
        else:
            # Find correct position
            current = self._head
            while current.next and current.next.priority <= priority:
                current = current.next
            new_node.next = current.next
            current.next = new_node

        self._size += 1

    def extract_min(self):
        """Remove and return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")

        item = self._head.item
        self._head = self._head.next
        self._size -= 1
        return item

    def peek_min(self):
        """Return minimum priority item. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")
        return self._head.item

    def is_empty(self):
        return self._head is None

    def size(self):
        return self._size

```

COMPARISON OF IMPLEMENTATIONS

Implementation	Insert	Extract Min	Peek Min	Space
Unsorted Array	O(1)	O(n)	O(n)	O(n)
Sorted Array	O(n)	O(1)	O(1)	O(n)
Sorted Linked List	O(n)	O(1)	O(1)	O(n)
Binary Heap	O(log n)	O(log n)	O(1)	O(n)

The binary heap implementation provides the best overall performance, which we'll explore in the next section.

9.4 Heaps

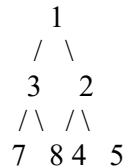
A heap uses a specialized tree-based data structure that satisfies the heap property. It's the most effective way to implement a priority queue.

Heap Properties:

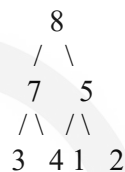
1. **Structure Property:** A heap is a complete binary tree (all levels filled except possibly the last, which is filled left-to-right)
2. **Order Property:** The priority of each node is greater than or equal to (max-heap) or less than or equal to (min-heap) its children's priorities

TYPES OF HEAPS

1. **Min-Heap:** Parent node has smaller priority than its children



2. **Max-Heap:** Parent node has larger priority than its children



HEAP REPRESENTATION

3. Heaps are typically implemented using arrays, providing efficient storage and easy navigation:

```

class MinHeap:
    """Array-based min-heap implementation."""

    def __init__(self):
        self._data = []

    def _parent_index(self, index):
        """Return parent index of given index."""
        return (index - 1) // 2

    def _left_child_index(self, index):
        """Return left child index."""
        return 2 * index + 1

    def _right_child_index(self, index):
        """Return right child index."""
        return 2 * index + 2

    def _has_parent(self, index):
        """Check if node has parent."""
        return self._parent_index(index) >= 0

    def _has_left_child(self, index):
        """Check if node has left child."""
        return self._left_child_index(index) < len(self._data)

```

```

def _has_right_child(self, index):
    """Check if node has right child."""
    return self._right_child_index(index) < len(self._data)

def _parent(self, index):
    """Get parent element."""
    return self._data[self._parent_index(index)]

def _left_child(self, index):
    """Get left child element."""
    return self._data[self._left_child_index(index)]

def _right_child(self, index):
    """Get right child element."""
    return self._data[self._right_child_index(index)]

def _swap(self, index1, index2):
    """Swap elements at two indices."""
    self._data[index1], self._data[index2] = self._data[index2], self._data[index1]

```

HEAP OPERATIONS

Insertion (Heapify Up): Insert at the end and bubble up to maintain heap property.

```

def insert(self, item, priority):
    """Insert item with priority. O(log n) time complexity."""
    self._data.append((priority, item))
    self._heapify_up(len(self._data) - 1)

def _heapify_up(self, index):
    """Restore heap property by moving element up."""
    while (self._has_parent(index) and
           self._parent(index)[0] > self._data[index][0]):
        parent_idx = self._parent_index(index)
        self._swap(index, parent_idx)
        index = parent_idx

```

Extraction (Heapify Down): Remove root, replace with last element, and bubble down.

```

def extract_min(self):
    """Remove and return minimum element. O(log n) time complexity."""
    if self.is_empty():
        raise ValueError("Heap is empty")

    # Store minimum element
    min_item = self._data[0][1]

    # Replace root with last element
    self._data[0] = self._data[-1]
    self._data.pop()

    # Restore heap property
    if not self.is_empty():

```

```

self._heapify_down(0)

return min_item

def _heapify_down(self, index):
    """Restore heap property by moving element down."""
    while self._has_left_child(index):
        # Find smaller child
        smaller_child_idx = self._left_child_index(index)
        if (self._has_right_child(index) and
            self._right_child(index)[0] < self._left_child(index)[0]):
            smaller_child_idx = self._right_child_index(index)

        # Check if swap is needed
        if self._data[index][0] < self._data[smaller_child_idx][0]:
            break

        # Swap and continue
        self._swap(index, smaller_child_idx)
        index = smaller_child_idx

```

Complete Min-Heap Implementation:

```

class MinHeapPriorityQueue:
    """Complete min-heap based priority queue implementation."""

    def __init__(self):
        self._data = []

    def _parent_index(self, index):
        return (index - 1) // 2

    def _left_child_index(self, index):
        return 2 * index + 1

    def _right_child_index(self, index):
        return 2 * index + 2

    def _has_parent(self, index):
        return self._parent_index(index) >= 0

    def _has_left_child(self, index):
        return self._left_child_index(index) < len(self._data)

    def _has_right_child(self, index):
        return self._right_child_index(index) < len(self._data)

    def _parent(self, index):
        return self._data[self._parent_index(index)]

    def _left_child(self, index):
        return self._data[self._left_child_index(index)]

    def _right_child(self, index):

```

```

    return self._data[self._right_child_index(index)]

    def _swap(self, index1, index2):
self._data[index1], self._data[index2] = self._data[index2], self._data[index1]

    def insert(self, item, priority):
        """Insert item with priority. O(log n) time complexity."""
self._data.append((priority, item))
self._heapify_up(len(self._data) - 1)

    def _heapify_up(self, index):
        """Restore heap property by moving element up."""
        while (self._has_parent(index) and
self._parent(index)[0] > self._data[index][0]):
parent_idx = self._parent_index(index)
self._swap(index, parent_idx)
        index = parent_idx

    def extract_min(self):
        """Remove and return minimum element. O(log n) time complexity."""
        if self.is_empty():
            raise ValueError("Heap is empty")

min_item = self._data[0][1]
self._data[0] = self._data[-1]
self._data.pop()

        if not self.is_empty():
self._heapify_down(0)

        return min_item

    def _heapify_down(self, index):
        """Restore heap property by moving element down."""
        while self._has_left_child(index):
smaller_child_idx = self._left_child_index(index)
            if (self._has_right_child(index) and
self._right_child(index)[0] < self._left_child(index)[0]):
smaller_child_idx = self._right_child_index(index)

            if self._data[index][0] < self._data[smaller_child_idx][0]:
                break

self._swap(index, smaller_child_idx)
        index = smaller_child_idx

    def peek_min(self):
        """Return minimum element without removing. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Heap is empty")
        return self._data[0][1]

    def is_empty(self):
        """Check if heap is empty. O(1) time complexity."""

```

```

        return len(self._data) == 0

    def size(self):
        """Return number of elements. O(1) time complexity."""
        return len(self._data)

# Example usage and demonstration
    def demonstrate_heap():
        """Demonstrate heap operations."""
        heap = MinHeapPriorityQueue()

# Insert elements
        tasks = [("High Priority Task", 1), ("Medium Task 1", 3),
                  ("Low Priority Task", 5), ("Medium Task 2", 3),
                  ("Critical Task", 0)]

        print("Inserting tasks:")
        for task, priority in tasks:
            heap.insert(task, priority)
        print(f"Inserted: {task} (Priority: {priority})")

        print(f"\nHeap size: {heap.size()}")
        print(f"Next task: {heap.peek_min()}")

        print("\nProcessing tasks in priority order:")
        while not heap.is_empty():
            task = heap.extract_min()
            print(f"Processing: {task}")

# Run demonstration
    demonstrate_heap()

```

BUILDING A HEAP FROM ARRAY

We can build a heap from an existing array using bottom-up heapify:

```

def build_heap(self, array_of_pairs):
    """Build heap from array of (priority, item) pairs. O(n) time complexity."""
    self._data = array_of_pairs.copy()

# Start from last non-leaf node and heapify down
    start_index = (len(self._data) - 2) // 2
    for i in range(start_index, -1, -1):
        self._heapify_down(i)

# Example usage
    heap = MinHeapPriorityQueue()
    data = [(4, "Task D"), (1, "Task A"), (3, "Task C"), (2, "Task B"), (5, "Task E")]
    heap.build_heap(data)

    print("Heap after building from array:")
    while not heap.is_empty():
        print(heap.extract_min())

```

9.5 Sorting with a Priority Queue

Heap sort uses a priority queue (heap) to sort elements efficiently. It has $O(n \log n)$ time complexity and sorts in-place.

Algorithm Steps:

1. Build a max-heap from the input array
2. Repeatedly extract the maximum element and place it at the end
3. Reduce the heap size and restore heap property

```
def heap_sort(arr):
    """Sort array using heap sort algorithm.  $O(n \log n)$  time complexity."""

    def heapify(arr, n, root):
        """Maintain max-heap property."""
        largest = root
        left = 2 * root + 1
        right = 2 * root + 2

        # Find largest among root and children
        if left < n and arr[left] > arr[largest]:
            largest = left
        if right < n and arr[right] > arr[largest]:
            largest = right

        # If largest is not root, swap and continue heapifying
        if largest != root:
            arr[root], arr[largest] = arr[largest], arr[root]
            heapify(arr, n, largest)

    n = len(arr)

    # Build max-heap (bottom-up)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        # Move current root to end
        arr[0], arr[i] = arr[i], arr[0]

        # Restore heap property for reduced heap
        heapify(arr, i, 0)

    return arr

# Example usage
def demonstrate_heap_sort():
    """Demonstrate heap sort algorithm."""
    arrays = [
        [64, 34, 25, 12, 22, 11, 90],
        [5, 2, 8, 1, 9],
        [1],
        []
    ]
```

```

    ]

    for arr in arrays:
        original = arr.copy()
    sorted_arr = heap_sort(arr)
    print(f"Original: {original}")
    print(f"Sorted: {sorted_arr}")
    print()

    demonstrate_heap_sort()

```

PRIORITY QUEUE SORT

We can also sort using a priority queue data structure:

```

def priority_queue_sort(arr):
    """Sort array using priority queue. O(n log n) time complexity."""
    pq = MinHeapPriorityQueue()

    # Insert all elements
    for i, value in enumerate(arr):
        pq.insert(value, value)

    # Extract elements in sorted order
    sorted_arr = []
    while not pq.is_empty():
        sorted_arr.append(pq.extract_min())

    return sorted_arr

# Example usage
original = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = priority_queue_sort(original)
print(f"Original: {original}")
print(f"Sorted: {sorted_arr}")

```

TOP-K ELEMENTS PROBLEM

Finding the K largest or smallest elements is a common application:

```

def find_k_largest(arr, k):
    """Find K largest elements using min-heap. O(n log k) time complexity."""
    if k <= 0 or k > len(arr):
        return []

    # Use min-heap of size k
    import heapq
    min_heap = []

    for num in arr:
        if len(min_heap) < k:
            heapq.heappush(min_heap, num)
        elif num > min_heap[0]:
            heapq.heapreplace(min_heap, num)

    return sorted(min_heap, reverse=True)

```

```

def find_k_smallest(arr, k):
    """Find K smallest elements using max-heap. O(n log k) time complexity."""
    if k <= 0 or k > len(arr):
        return []

    # Use max-heap of size k (negate values for max-heap with min-heap)
    import heapq
    max_heap = []

    for num in arr:
        if len(max_heap) < k:
            heapq.heappush(max_heap, -num)
        elif num < -max_heap[0]:
            heapq.heapreplace(max_heap, -num)

    return sorted([-x for x in max_heap])

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90, 88, 76, 50]
print(f'Array: {arr}')
print(f'3 largest: {find_k_largest(arr, 3)}')
print(f'3 smallest: {find_k_smallest(arr, 3)}')

```

9.6 Adaptable Priority Queues

CONCEPT AND MOTIVATION

Adaptable priority queues support additional operations to modify existing entries:

- Change the priority of an existing item
- Remove a specific item from the queue

These operations are essential in applications like Dijkstra's algorithm where priorities need to be updated dynamically.

IMPLEMENTATION WITH LOCATION-AWARE ENTRIES

```

class Entry:
    """Entry class for adaptable priority queue."""
    def __init__(self, key, value, index):
        self._key = key    # priority
        self._value = value # item
        self._index = index # current index in heap

    def __lt__(self, other):
        return self._key < other._key

class AdaptablePriorityQueue:
    """Adaptable priority queue using binary heap."""

    def __init__(self):
        self._data = []
        self._entries = {} # Maps values to entries for fast lookup

```

```

def _parent(self, j):
    return (j - 1) // 2

def _left(self, j):
    return 2 * j + 1

def _right(self, j):
    return 2 * j + 2

def _has_left(self, j):
    return self._left(j) < len(self._data)

def _has_right(self, j):
    return self._right(j) < len(self._data)

def _swap(self, i, j):
    """Swap entries and update their indices."""
    self._data[i], self._data[j] = self._data[j], self._data[i]
    self._data[i]._index = i
    self._data[j]._index = j

def _upheap(self, j):
    """Restore heap property by moving entry up."""
    parent = self._parent(j)
    if j > 0 and self._data[j] < self._data[parent]:
        self._swap(j, parent)
        self._upheap(parent)

def _downheap(self, j):
    """Restore heap property by moving entry down."""
    if self._has_left(j):
        left = self._left(j)
        small_child = left
        if self._has_right(j):
            right = self._right(j)
            if self._data[right] < self._data[left]:
                small_child = right
        if self._data[small_child] < self._data[j]:
            self._swap(j, small_child)
            self._downheap(small_child)

def insert(self, key, value):
    """Insert new entry. O(log n) time complexity."""
    if value in self._entries:
        raise ValueError(f"Value {value} already exists")

    entry = Entry(key, value, len(self._data))
    self._data.append(entry)
    self._entries[value] = entry
    self._upheap(len(self._data) - 1)
    return entry

def min(self):

```

```

        """Return minimum entry without removing. O(1) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")
        return (self._data[0]._key, self._data[0]._value)

    def remove_min(self):
        """Remove and return minimum entry. O(log n) time complexity."""
        if self.is_empty():
            raise ValueError("Priority queue is empty")

    # Store minimum entry
    min_entry = self._data[0]
    min_key, min_value = min_entry._key, min_entry._value

    # Replace with last entry
    last_entry = self._data[-1]
    self._data[0] = last_entry
    last_entry._index = 0
    self._data.pop()

    # Remove from entries map
    del self._entries[min_value]

    # Restore heap property
    if not self.is_empty():
        self._downheap(0)

    return (min_key, min_value)

    def update(self, value, new_key):
        """Update priority of existing entry. O(log n) time complexity."""
        if value not in self._entries:
            raise ValueError(f"Value {value} not found")

        entry = self._entries[value]
        old_key = entry._key
        entry._key = new_key

    # Restore heap property
    if new_key < old_key:
        self._upheap(entry._index)
    elif new_key > old_key:
        self._downheap(entry._index)

    def remove(self, value):
        """Remove specific entry. O(log n) time complexity."""
        if value not in self._entries:
            raise ValueError(f"Value {value} not found")

        entry = self._entries[value]
        j = entry._index

        if j == len(self._data) - 1:
            # Last element, just remove

```

```

self._data.pop()
    else:
        # Replace with last element
        last_entry = self._data[-1]
        self._data[j] = last_entry
        last_entry._index = j
        self._data.pop()

        # Restore heap property
        self._upheap(j)
        self._downheap(j)

        # Remove from entries map
        del self._entries[value]
        return (entry._key, value)

    def is_empty(self):
        return len(self._data) == 0

    def size(self):
        return len(self._data)

    # Example usage and demonstration
    def demonstrate_adaptable_pq():
        """Demonstrate adaptable priority queue operations."""
        pq = AdaptablePriorityQueue()

        # Insert tasks
        tasks = [("Task A", 3), ("Task B", 1), ("Task C", 4), ("Task D", 2)]

        print("Inserting tasks:")
        for task, priority in tasks:
            pq.insert(priority, task)
        print(f"Inserted {task} with priority {priority}")

        print(f"\nCurrent minimum: {pq.min()}")

        # Update priority
        print("\nUpdating Task C priority from 4 to 0:")
        pq.update("Task C", 0)
        print(f"New minimum: {pq.min()}")

        # Remove specific task
        print("\nRemoving Task A:")
        removed = pq.remove("Task A")
        print(f"Removed: {removed}")

        print("\nProcessing remaining tasks:")
        while not pq.is_empty():
            task = pq.

```

1. Question: Which of the following is NOT a type of tree traversal? a) Pre-order traversal b) In-order traversal c) Sequential traversal d) Post-order traversal
a. Answer: c) Sequential traversal
2. Question: Give the highest possible nodes at level 3 (assuming root is at level 0)? a) 4 b) 8 c) 16 d) 2
a. Answer: b) 8
3. Question: A binary tree with 10 nodes has how many NULL links (empty child references)? a) 9 b) 10 c) 11 d) 12
a. Answer: c) 11
4. Question: Which type of binary tree has all internal nodes containing exactly two children? a) Complete binary tree b) Full binary tree c) Perfect binary tree d) Balanced binary tree Answer: b) Full binary tree
5. Question: What is the order of visiting nodes in a pre-order traversal? a) Root, Right, Left b) Left, Root, Right c) Root, Left, Right d) Left, Right, Root
a. Answer: c) Root, Left, Right
6. Question: In an expression tree, where are the operands located? a) Only at the root b) Only at leaf nodes c) At any internal node d) Only at the second level
a. Answer: b)
7. Question: What would be the level-order traversal of the following binary tree?
Copy
1
 /\n 2 3
 /\n 4 5
a) 1, 2, 3, 4, 5 b) 1, 3, 2, 5, 4 c) 4, 2, 5, 1, 3 d) 1, 2, 3, 5, 4
a. Answer: a) 1, 2, 3, 4, 5
8. Question: A complete binary tree with 7 nodes has which height? a) 2 b) 3 c) 4 d) 5
a. Answer: a) 2
9. Question: Which data structure is typically used to implement level-order traversal? a) Stack b) Queue c) Linked List d) Array
a. Answer: b) Queue
10. Question: What is the time complexity of inserting a node into a binary tree using level-order insertion? a) $O(1)$ b) $O(\log n)$ c) $O(n)$ d) $O(n^2)$
a. Answer: c) $O(n)$

9.7 Summary

- Priority queues are fundamental data structures that extend regular queues by serving elements based on their priority rather than insertion order. This unit covered the essential aspects of priority queues and their most efficient implementation using heaps.

Key Concepts Covered:

- **Priority Queue Abstract Data Type:** Understanding the basic operations (insert, extract_min/max, peek) and their applications in scheduling, graph algorithms, and simulation systems.

- **Implementation Strategies:** We explored various approaches including unsorted arrays, sorted arrays, linked lists, and binary heaps, analyzing their time complexity trade-offs.
- **Binary Heaps:** The most efficient priority queue implementation, providing $O(\log n)$ insertion and extraction with $O(1)$ peek operations. We covered both min-heaps and max-heaps, their structural and ordering properties.
- **Heap Operations:** Detailed implementation of heapify-up and heapify-down operations, array-based representation, and bottom-up heap construction.
- **Sorting Applications:** Heap sort algorithm demonstrating how priority queues can achieve $O(n \log n)$ sorting performance, along with solutions to top-K problems.
- **Adaptable Priority Queues:** Advanced implementations supporting priority updates and specific element removal, essential for dynamic algorithms like Dijkstra's shortest path.

Performance Summary:

- Standard heap operations: $O(\log n)$ for insertion and extraction
- Heap construction from array: $O(n)$ time complexity
- Heap sort: $O(n \log n)$ time, $O(1)$ extra space
- Adaptable operations: $O(\log n)$ for updates and removals
- Priority queues find extensive applications in operating systems, network protocols, graph algorithms, and data processing tasks. Mastering these concepts provides a solid foundation for advanced algorithms and system design.

9.8 Solutions / Answers to Check Your Progress

1. Question: Which of the following is NOT a standard priority queue operation?
a) insert(item, priority) b) extract_min() c) sort_all() d) peek()

Answer: c) sort_all()

2. Question: In a min-heap with 15 elements, what is the maximum number of comparisons needed to insert a new element? a) 3 b) 4 c) 5 d) 15

Answer: b) 4

3. Question: Which array represents a valid max-heap? a) [90, 80, 70, 60, 50, 40, 30] b) [90, 70, 80, 60, 50, 30, 40] c) [90, 80, 85, 60, 70, 40, 30] d) [30, 40, 50, 60, 70, 80, 90]

Answer: c) [90, 80, 85, 60, 70, 40, 30]

4. Question: What is the time complexity of building a heap from an unsorted array using the bottom-up approach? a) $O(n \log n)$ b) $O(n^2)$ c) $O(n)$ d) $O(\log n)$

Answer: c) $O(n)$

•

5. Question: In heap sort, after building the initial max-heap, what operation is repeatedly performed? a) Insert the maximum element b) Extract maximum and heapify c) Search for minimum element d) Rebuild the entire heap

Answer: b) Extract maximum and heapify

6. Question: Which implementation provides the best overall time complexity for priority queue operations? a) Unsorted array b) Sorted array c) Binary heap d) Sorted linked list

- **Answer:** c) Binary heap
-

7. Question: In an adaptable priority queue, what additional capability is provided? a) Faster insertion b) Multiple priority levels c) Modifying existing entries d) Automatic sorting

Answer: c) Modifying existing entries

8. Question: What is the space complexity of the heap sort algorithm? a) $O(n)$ b) $O(\log n)$ c) $O(1)$ d) $O(n \log n)$

Answer: c) $O(1)$

9. Question: In a binary heap stored as an array, if a node is at index i , where is its left child? a) $2i$ b) $2i + 1$ c) $i + 1$ d) $i/2$

Answer: b) $2i + 1$

- **10. Question:** Which of the following applications would benefit most from using a priority queue? a) FIFO task processing b) CPU process scheduling c) Stack-based calculations d) Sequential data storage

Answer: b) CPU process scheduling

9.9 Further Readings

<https://www.geeksforgeeks.org/dsa/priority-queue-set-1-introduction/>

UNIT 10 MAPS, HASH TABLES, AND SKIP LISTS

Structure

10.0	Introduction
10.1	Objectives
10.2	Maps and Dictionaries,
10.3	Hash Tables,
10.4	Sorted Maps,
10.5	Skip Lists, Sets,
10.6	Multisets, and Multimaps.
10.7	Summary
10.8	Solutions/ Answers to Check your Progress
10.9	Further Readings

10.0 Introduction

Maps and dictionaries are fundamental data structures that store key-value pairs, enabling efficient data retrieval based on unique identifiers. These structures are essential for applications requiring fast lookups, such as databases, caching systems, symbol tables in compilers, and associative arrays in programming languages.

The efficiency of map operations depends heavily on the underlying implementation. Hash tables provide average-case $O(1)$ operations, while balanced trees like AVL or Red-Black trees guarantee $O(\log n)$ worst-case performance. Advanced structures like skip lists offer probabilistic performance guarantees and simpler implementation compared to balanced trees.

This unit explores various implementations of the map abstract data type, from basic hash tables to advanced structures like skip lists, sets, multisets, and multimaps. Each concept is illustrated with detailed implementations, complexity analysis, and practical applications.

10.1 Objectives

After completing this unit, you will be able to:

- Understand the map abstract data type and its fundamental operations
- Implement various map data structures including hash tables with different collision resolution strategies
- Design and implement hash functions for different data types
- Create sorted maps using balanced binary search trees
- Implement skip lists as probabilistic alternative to balanced trees Work with set data structures and understand their relationship to maps
- Implement multisets and multimaps for handling duplicate keys/values
- Analyze time and space complexity of different map implementations
- Choose appropriate map implementation based on application requirements

10.2 Maps and Dictionaries

DEFINITION AND ABSTRACT DATA TYPE

A map (also called dictionary or associative array) is an abstract data type that stores key-value pairs, where each key is unique and maps to exactly one value. Maps provide efficient access to values based on their associated keys.

Map Structure Visualization:

```
Map: {
    key1 → value1
    key2 → value2
    key3 → value3
    ...
    keyn → valuen
}
```

FUNDAMENTAL MAP OPERATIONS

The essential operations for a map include:

Primary Operations:

- `put(key, value)`: Insert or update a key-value pair
- `get(key)`: Retrieve the value associated with a key
- `remove(key)`: Remove a key-value pair
- `contains(key)`: Check if a key exists in the map
- `is_empty()`: Check if the map is empty
- `size()`: Return the number of key-value pairs

Additional Operations:

- `keys()`: Return all keys in the map
- `values()`: Return all values in the map
- `items()`: Return all key-value pairs

SIMPLE LIST-BASED MAP IMPLEMENTATION

A basic implementation using a list of key-value pairs:

```
class ListMap(MapBase):
    """Simple map implementation using list of key-value pairs."""

    def __init__(self):
        """Initialize empty map."""
        self._data = []

    def put(self, key, value):
        """Insert or update key-value pair. O(n) time complexity."""
        # Check if key already exists
        for i, (k, v) in enumerate(self._data):
            if k == key:
                self._data[i] = (key, value) # Update existing
                return

        # Add new key-value pair
        self._data.append((key, value))

    def get(self, key):
        """Retrieve value for key. O(n) time complexity."""
        for k, v in self._data:
            if k == key:
```

```

        return v
    raise KeyError(f'Key '{key}' not found")

def remove(self, key):
    """Remove key-value pair. O(n) time complexity."""
    for i, (k, v) in enumerate(self._data):
        if k == key:
            value = self._data.pop(i)
            return value[1]
    raise KeyError(f'Key '{key}' not found")

def contains(self, key):
    """Check if key exists. O(n) time complexity."""
    for k, v in self._data:
        if k == key:
            return True
    return False

def is_empty(self):
    """Check if map is empty. O(1) time complexity."""
    return len(self._data) == 0

def size(self):
    """Return number of pairs. O(1) time complexity."""
    return len(self._data)

def keys(self):
    """Return iterator over all keys."""
    return (k for k, v in self._data)

def values(self):
    """Return iterator over all values."""
    return (v for k, v in self._data)

def items(self):
    """Return iterator over all key-value pairs."""
    return iter(self._data)

def __iter__(self):
    """Make map iterable over keys."""
    return self.keys()

def __repr__(self):
    """String representation of map."""
    pairs = [f'{k}: {v}' for k, v in self._data]
    return "{" + ", ".join(pairs) + "}"

# Demonstration of ListMap
def demonstrate_list_map():
    """Demonstrate basic map operations."""
    print("=== ListMap Demonstration ===")

    # Create and populate map
    student_grades = ListMap()

```

```

student_grades.put("Alice", 85)
student_grades.put("Bob", 92)
student_grades.put("Charlie", 78)
student_grades.put("Diana", 96)

print(f'Initial map: {student_grades}')
print(f'Size: {student_grades.size()}')

# Access values
print(f'Alice's grade: {student_grades.get('Alice')}')
print(f'Bob in map: {'Bob' in student_grades}')

# Update existing value
student_grades.put("Alice", 88)
print(f'After updating Alice: {student_grades}')

# Remove entry
removed_grade = student_grades.remove("Charlie")
print(f'Removed Charlie (grade: {removed_grade})')
print(f'Final map: {student_grades}')

# Iterate over map
print("All students and grades:")
for student in student_grades:
    print(f' {student}: {student_grades[student]}')

demonstrate_list_map()

```

APPLICATIONS OF MAPS

Maps have numerous practical applications:

Database Systems:

- Primary key to record mapping
- Index structures for fast data retrieval

Compiler Design:

- Symbol tables mapping variable names to memory locations
- Keyword recognition and token mapping

Web Applications:

- Session management (session ID to user data)
- URL routing (path to handler function)

Caching Systems:

- Key-value stores for frequently accessed data
- Content Delivery Networks (CDN) cache mapping

Configuration Management:

- Application settings and preferences
- Environment variable storage

10.3 Hash Tables

HASH TABLE FUNDAMENTALS

A hash table uses a hash function to map keys to indices in an array, providing average-case $O(1)$ time complexity for basic operations. The hash function distributes keys uniformly across the array to minimize collisions.

Hash Table Structure:Hash Function: key $\rightarrow$ indexArray: [bucket₀, bucket₁, bucket₂, ..., bucket_{n-1}]

Example:

hash("apple") = 3 $\rightarrow$ store at index 3hash("banana") = 7 $\rightarrow$ store at index 7hash("cherry") = 3 $\rightarrow$ collision at index 3!**HASH FUNCTIONS**

A good hash function should:

- Distribute keys uniformly across the hash table
- Be deterministic (same key always produces same hash)
- Be efficient to compute
- Minimize collisions

Common Hash Functions:

```

class HashFunctions:
    """Collection of hash functions for different data types."""

    @staticmethod
    def hash_string_simple(key, table_size):
        """Simple string hash using sum of ASCII values."""
        hash_value = 0
        for char in key:
            hash_value += ord(char)
        return hash_value % table_size

    @staticmethod
    def hash_string_polynomial(key, table_size, base=31):
        """Polynomial rolling hash for strings."""
        hash_value = 0
        for i, char in enumerate(key):
            hash_value += ord(char) * (base ** i)
        return hash_value % table_size

    @staticmethod
    def hash_string_djb2(key, table_size):
        """DJB2 hash algorithm for strings."""
        hash_value = 5381
        for char in key:
            hash_value = ((hash_value << 5) + hash_value) + ord(char)
        return hash_value % table_size

    @staticmethod
    def hash_integer_division(key, table_size):
        """Simple modular hash for integers."""
        return abs(key) % table_size

    @staticmethod
    def hash_integer_multiplication(key, table_size, A=0.6180339887):
        """Multiplication method for integer hashing."""
        return int(table_size * ((abs(key) * A) % 1))

# Demonstrate hash functions
def demonstrate_hash_functions():

```

```

"""Compare different hash functions."""
print("=== Hash Function Comparison ===")

keys = ["apple", "banana", "cherry", "date", "elderberry"]
table_size = 10

print(f"Table size: {table_size}")
print(f"Keys: {keys}")
print()

# Simple sum hash
print("Simple Sum Hash:")
for key in keys:
    hash_val = HashFunctions.hash_string_simple(key, table_size)
    print(f"hash('{key}') = {hash_val}")

print("\nPolynomial Hash:")
for key in keys:
    hash_val = HashFunctions.hash_string_polynomial(key, table_size)
    print(f"hash('{key}') = {hash_val}")

print("\nDJB2 Hash:")
for key in keys:
    hash_val = HashFunctions.hash_string_djb2(key, table_size)
    print(f"hash('{key}') = {hash_val}")

demonstrate_hash_functions()

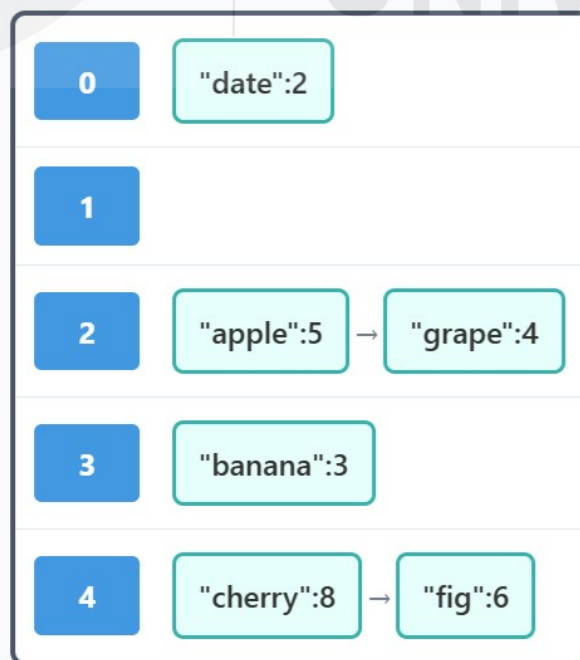
```

COLLISION RESOLUTION STRATEGIES

Sometimes hash functions maps multiple keys to same index that requires collision resolution -

SEPARATE CHAINING

In this method, we create buckets of key value pairs that has to same index as shown below -



```

class ChainHashMap(MapBase):
    """Hash table implementation using separate chaining."""

    def __init__(self, capacity=11, hash_function=None):
        """Initialize hash table with given capacity."""
        self._capacity = capacity
        self._size = 0
        self._buckets = [[] for _ in range(capacity)]
        self._hash_function = hash_function or self._default_hash

    def _default_hash(self, key):
        """Default hash function using Python's built-in hash."""
        return hash(key) % self._capacity

    def _bucket_get_item(self, bucket, key):
        """Search for key in bucket, return (index, value) or (None, None)."""
        for i, (k, v) in enumerate(bucket):
            if k == key:
                return i, v
        return None, None

    def put(self, key, value):
        """Insert or update key-value pair. Average O(1) time."""
        bucket_index = self._hash_function(key)
        bucket = self._buckets[bucket_index]

        # Check if key already exists
        item_index, old_value = self._bucket_get_item(bucket, key)
        if item_index is not None:
            # Update existing key
            bucket[item_index] = (key, value)
        else:
            # Add new key-value pair
            bucket.append((key, value))
        self._size += 1

        # Check if resize is needed
        if self._size > self._capacity:
            self._resize()

    def get(self, key):
        """Retrieve value for key. Average O(1) time."""
        bucket_index = self._hash_function(key)
        bucket = self._buckets[bucket_index]

        item_index, value = self._bucket_get_item(bucket, key)
        if item_index is not None:
            return value
        raise KeyError(f'Key '{key}' not found')

    def remove(self, key):
        """Remove key-value pair. Average O(1) time."""

```

```

bucket_index = self._hash_function(key)
bucket = self._buckets[bucket_index]

item_index, value = self._bucket_get_item(bucket, key)
if item_index is not None:
    bucket.pop(item_index)
self._size -= 1
    return value
    raise KeyError(f'Key '{key}' not found")

def contains(self, key):
    """Check if key exists. Average O(1) time."""
    try:
        self.get(key)
        return True
    except KeyError:
        return False

def is_empty(self):
    """Check if map is empty."""
    return self._size == 0

def size(self):
    """Return number of key-value pairs."""
    return self._size

def _resize(self):
    """Resize hash table when load factor becomes too high."""
    old_buckets = self._buckets
    self._capacity = 2 * self._capacity + 1 # Keep it odd for better distribution
    self._size = 0
    self._buckets = [[] for _ in range(self._capacity)]

    # Rehash all existing items
    for bucket in old_buckets:
        for key, value in bucket:
            self.put(key, value)

def load_factor(self):
    """Return current load factor."""
    return self._size / self._capacity

def bucket_distribution(self):
    """Return distribution of items across buckets for analysis."""
    distribution = []
    for i, bucket in enumerate(self._buckets):
        if bucket:
            distribution.append((i, len(bucket)))
    return distribution

def keys(self):
    """Return iterator over all keys."""
    for bucket in self._buckets:

```

```

        for key, value in bucket:
            yield key

def values(self):
    """Return iterator over all values."""
    for bucket in self._buckets:
        for key, value in bucket:
            yield value

def items(self):
    """Return iterator over all key-value pairs."""
    for bucket in self._buckets:
        for item in bucket:
            yield item

def __repr__(self):
    """String representation showing internal structure."""
    result = f'ChainHashMap(capacity={self._capacity}, size={self._size})\n'
    for i, bucket in enumerate(self._buckets):
        if bucket:
            result += f' Bucket {i}: {bucket}\n'
    return result

# Demonstrate chain hash map
def demonstrate_chain_hash_map():
    """Demonstrate hash table with separate chaining."""
    print("==== Chain Hash Map Demonstration =====")

    # Create hash map
    hash_map = ChainHashMap(capacity=7)

    # Add items
    items = [("apple", 5), ("banana", 3), ("cherry", 8), ("date", 2),
             ("elderberry", 1), ("fig", 6), ("grape", 4)]

    print("Adding items:")
    for key, value in items:
        hash_map.put(key, value)
        print(f' Added '{key}': {value}')

    print(f'\nHash map structure:')
    print(hash_map)

    print(f'Load factor: {hash_map.load_factor():.2f}')
    print(f'Bucket distribution: {hash_map.bucket_distribution()}')

    # Test operations
    print(f'\nTesting operations:')
    print(f'Get 'banana': {hash_map.get('banana')}')
    print(f'Contains 'cherry': {hash_map.contains('cherry')}')

    # Remove item
    removed = hash_map.remove('date')
    print(f'Removed 'date': {removed}')

```

```
# Show final state
print(f"\nFinal size: {hash_map.size()}")
print("All items:", dict(hash_map.items()))
```

```
demonstrate_chain_hash_map()
```

OPEN ADDRESSING (LINEAR PROBING)

Store all items directly in the hash table array, using probing to find alternative locations.

```
class ProbeHashMap(MapBase):
    """Hash table implementation using linear probing."""

    _AVAIL = object() # Sentinel for deleted items

    def __init__(self, capacity=11):
        """Initialize hash table with given capacity."""
        self._capacity = capacity
        self._size = 0
        self._data = [None] * capacity

    def _hash(self, key):
        """Hash function using Python's built-in hash."""
        return hash(key) % self._capacity

    def _is_available(self, j):
        """Return True if index j is available (None or deleted)."""
        return self._data[j] is None or self._data[j] is ProbeHashMap._AVAIL

    def _find_slot(self, key):
        """Find slot for key using linear probing.

        Returns:
            (found, index): found=True if key exists, index of key/available slot
        """
        avail = None
        j = self._hash(key)

        while not self._is_available(j):
            if self._data[j][0] == key:
                return True, j # Found existing key
            if avail is None and self._data[j] is ProbeHashMap._AVAIL:
                avail = j # Remember first available slot
            j = (j + 1) % self._capacity

        # Key not found, return available slot
        return False, avail if avail is not None else j

    def put(self, key, value):
        """Insert or update key-value pair. Average O(1) time."""
        found, j = self._find_slot(key)

        if found:
            # Update existing key
```

```

self._data[j] = (key, value)
    else:
        # Insert new key
self._data[j] = (key, value)
self._size += 1

        # Check if resize is needed (load factor > 0.5)
        if self._size > self._capacity // 2:
self._resize(2 * self._capacity + 1)

def get(self, key):
    """Retrieve value for key. Average O(1) time."""
    found, j = self._find_slot(key)
    if found:
        return self._data[j][1]
    raise KeyError(f'Key '{key}' not found")

def remove(self, key):
    """Remove key-value pair. Average O(1) time."""
    found, j = self._find_slot(key)
    if found:
        value = self._data[j][1]
self._data[j] = ProbeHashMap._AVAIL # Mark as deleted
self._size -= 1
        return value
    raise KeyError(f'Key '{key}' not found")

def contains(self, key):
    """Check if key exists. Average O(1) time."""
    found, j = self._find_slot(key)
    return found

def is_empty(self):
    """Check if map is empty."""
    return self._size == 0

def size(self):
    """Return number of key-value pairs."""
    return self._size

def _resize(self, new_capacity):
    """Resize hash table to new capacity."""
old_data = self._data
self._capacity = new_capacity
self._size = 0
self._data = [None] * new_capacity

    # Rehash all existing items
    for item in old_data:
        if item is not None and item is not ProbeHashMap._AVAIL:
self.put(item[0], item[1])

def load_factor(self):
    """Return current load factor."""

```

```

        return self._size / self._capacity

    def keys(self):
        """Return iterator over all keys."""
        for item in self._data:
            if item is not None and item is not ProbeHashMap._AVAIL:
                yield item[0]

    def values(self):
        """Return iterator over all values."""
        for item in self._data:
            if item is not None and item is not ProbeHashMap._AVAIL:
                yield item[1]

    def items(self):
        """Return iterator over all key-value pairs."""
        for item in self._data:
            if item is not None and item is not ProbeHashMap._AVAIL:
                yield item

    def __repr__(self):
        """String representation showing internal structure."""
        result = f"ProbeHashMap(capacity={self._capacity}, size={self._size})\n"
        for i, item in enumerate(self._data):
            if item is not None:
                if item is ProbeHashMap._AVAIL:
                    result += f" [{i}]: <DELETED>\n"
                else:
                    result += f" [{i}]: {item}\n"
        return result

# Demonstrate probe hash map
def demonstrate_probe_hash_map():
    """Demonstrate hash table with linear probing."""
    print("=== Probe Hash Map Demonstration ===")

    # Create hash map
    hash_map = ProbeHashMap(capacity=7)

    # Add items (some will cause collisions)
    items = [("apple", 5), ("banana", 3), ("cherry", 8), ("date", 2)]

    print("Adding items:")
    for key, value in items:
        hash_map.put(key, value)
        print(f" Added '{key}': {value}")

    print(f"\nHash map structure:")
    print(hash_map)

    print(f"Load factor: {hash_map.load_factor():.2f}")

# Test collision handling
print("Testing collision handling:")

```

```

hash_map.put("elderberry", 1)
hash_map.put("fig", 6)

print(f"\nAfter adding more items:")
print(hash_map)

# Test removal
print("Testing removal:")
removed = hash_map.remove('banana')
print(f'Removed 'banana': {removed}')

print(f"\nAfter removal:")
print(hash_map)

demonstrate_probe_hash_map()

```

QUADRATIC PROBING AND DOUBLE HASHING

Alternative probing strategies to reduce clustering:

```

class AdvancedProbeHashMap(ProbeHashMap):
    """Hash table with quadratic probing and double hashing options."""

    def __init__(self, capacity=11, probe_type='linear'):
        """Initialize with specified probing strategy."""
        super().__init__(capacity)
        self._probe_type = probe_type

    def _probe_sequence(self, key):
        """Generate probe sequence based on strategy."""
        h1 = hash(key) % self._capacity

        if self._probe_type == 'linear':
            # Linear probing: h1, h1+1, h1+2, ...
            for i in range(self._capacity):
                yield (h1 + i) % self._capacity

        elif self._probe_type == 'quadratic':
            # Quadratic probing: h1, h1+1^2, h1+2^2, ...
            for i in range(self._capacity):
                yield (h1 + i*i) % self._capacity

        elif self._probe_type == 'double':
            # Double hashing: h1, h1+h2, h1+2*h2, ...
            h2 = 7 - (hash(key) % 7) # Second hash function
            for i in range(self._capacity):
                yield (h1 + i * h2) % self._capacity

    def _find_slot(self, key):
        """Find slot using advanced probing strategy."""
        avail = None

        for j in self._probe_sequence(key):
            if self._data[j] is None:
                # Empty slot found
                return False, avail if avail is not None else j

        elif self._data[j] is ProbeHashMap._AVAIL:

```

```

        # Deleted slot found
        if avail is None:
            avail = j
    elif self._data[j][0] == key:
        # Key found
        return True, j

    # Should never reach here with proper capacity management
    raise Exception("Hash table is full")

# Demonstrate different probing strategies
def compare_probing_strategies():
    """Compare different probing strategies."""
    print("=== Probing Strategy Comparison ===")

    strategies = ['linear', 'quadratic', 'double']
    keys = ["apple", "banana", "cherry", "date", "elderberry"]

    for strategy in strategies:
        print(f"\n{strategy.title()} Probing:")
        hash_map = AdvancedProbeHashMap(capacity=11, probe_type=strategy)

        for key in keys:
            hash_map.put(key, len(key))

        print(f"Final structure:")
        print(hash_map)

    compare_probing_strategies()

```

HASH TABLE PERFORMANCE ANALYSIS

Time Complexity:

- Average case: $O(1)$ for all operations
- Worst case: $O(n)$ when all keys hash to same bucket (chaining) or when many collisions occur (probing)

Space Complexity: $O(n)$ where n is the number of key-value pairs

Load Factor Impact:

- Chaining: Performance degrades gradually as load factor increases
- Open Addressing: Performance degrades rapidly when load factor > 0.7

10.4 Sorted Maps

SORTED MAP FUNDAMENTALS

A sorted map maintains key-value pairs in sorted order by key, enabling additional operations like range queries and ordered iteration. Sorted maps are typically implemented using balanced binary search trees.

Sorted Map Additional Operations:

- `first_key()`: Return smallest key
- `last_key()`: Return largest key
- `before(key)`: Return largest key smaller than given key
- `after(key)`: Return smallest key larger than given key
- `range(start, stop)`: Return all keys in range $[start, stop)$

BINARY SEARCH TREE IMPLEMENTATION

```

class BSTNode:
    """Node class for binary search tree."""

    def __init__(self, key, value, left=None, right=None, parent=None):
self.key = key
self.value = value
self.left = left
self.right = right
self.parent = parent

    def __repr__(self):
        return f"BSTNode({self.key}: {self.value})"

class BinarySearchTreeMap(MapBase):
    """Sorted map implementation using binary search tree."""

    def __init__(self):
        """Initialize empty BST."""
self._root = None
self._size = 0

    def _subtree_search(self, node, key):
        """Search for key in subtree rooted at node."""
        if node is None or key == node.key:
            return node
        elif key < node.key:
            return self._subtree_search(node.left, key)
        else:
            return self._subtree_search(node.right, key)

    def _subtree_min(self, node):
        """Return node with minimum key in subtree."""
        while node.left is not None:
            node = node.left
        return node

    def _subtree_max(self, node):
        """Return node with maximum key in subtree."""
        while node.right is not None:
            node = node.right
        return node

    def put(self, key, value):
        """Insert or update key-value pair. O(h) time where h is height."""
        if self._root is None:
self._root = BSTNode(key, value)
self._size += 1
        else:
            self._insert_node(self._root, key, value)

    def _insert_node(self, node, key, value):
        """Helper method to insert node."""
        if key == node.key:
            # Update existing key

```

```

node.value = value
elif key < node.key:
    if node.left is None:
        node.left = BSTNode(key, value, parent=node)
    self._size += 1
    else:
        self._insert_node(node.left, key, value)
    else:
        if node.right is None:
            node.right = BSTNode(key, value, parent=node)
            self._size += 1
        else:
            self._insert_node(node.right, key, value)

def get(self, key):
    """Retrieve value for key. O(h) time."""
    node = self._subtree_search(self._root, key)
    if node is not None:
        return node.value
    raise KeyError(f'Key '{key}' not found')

def remove(self, key):
    """Remove key-value pair. O(h) time."""
    node = self._subtree_search(self._root, key)
    if node is None:
        raise KeyError(f'Key '{key}' not found')

    value = node.value
    self._delete_node(node)
    self._size -= 1
    return value

def _delete_node(self, node):
    """Delete node from BST."""
    if node.left is None and node.right is None:
        # Case 1: Leaf node
        self._replace_node(node, None)
    elif node.left is None:
        # Case 2: Only right child
        self._replace_node(node, node.right)
    elif node.right is None:
        # Case 2: Only left child
        self._replace_node(node, node.left)
    else:
        # Case 3: Two children - replace with inorder successor
        successor = self._subtree_min(node.right)
        node.key = successor.key
        node.value = successor.value
        self._delete_node(successor) # Delete successor (has at most one child)

def _replace_node(self, node, replacement):
    """Replace node with replacement in the tree."""
    if node.parent is None:

```

```

        # Replacing root
self._root = replacement
elif node == node.parent.left:
node.parent.left = replacement
    else:
node.parent.right = replacement

    if replacement is not None:
replacement.parent = node.parent

def contains(self, key):
    """Check if key exists. O(h) time."""
    return self._subtree_search(self._root, key) is not None

def is_empty(self):
    """Check if map is empty."""
    return self._size == 0

def size(self):
    """Return number of key-value pairs."""
    return self._size

def first_key(self):
    """Return smallest key. O(h) time."""
    if self._root is None:
        raise ValueError("Map is empty")
    return self._subtree_min(self._root).key

def last_key(self):
    """Return largest key. O(h) time."""
    if self._root is None:
        raise ValueError("Map is empty")
    return self._subtree_max(self._root).key

def before(self, key):
    """Return largest key smaller than given key."""
    node = self._root
    predecessor = None

    while node is not None:
        if key <= node.key:
            node = node.left
        else:
            predecessor = node
            node = node.right

    if predecessor is not None:
        return predecessor.key
    raise ValueError("No key before the given key")

def after(self, key):
    """Return smallest key larger than given key."""
    node = self._root
    successor = None

```

```

while node is not None:
    if key >= node.key:
        node = node.right
    else:
        successor = node
        node = node.left

if successor is not None:
    return successor.key
raise ValueError("No key after the given key")

def range(self, start, stop):
    """Return iterator over keys in range [start, stop)."""
    def _range_helper(node):
        if node is not None:
            if start <= node.key:
                yield from _range_helper(node.left)
            if start <= node.key < stop:
                yield node.key
            if node.key < stop:
                yield from _range_helper(node.right)

    return _range_helper(self._root)

def keys(self):
    """Return iterator over all keys in sorted order."""
    return self._inorder_keys(self._root)

def _inorder_keys(self, node):
    """Inorder traversal to get keys in sorted order."""
    if node is not None:
        yield from self._inorder_keys(node.left)
        yield node.key
        yield from self._inorder_keys(node.right)

def values(self):
    """Return iterator over all values in key order."""
    return self._inorder_values(self._root)

def _inorder_values(self, node):
    """Inorder traversal to get values."""
    if node is not None:
        yield from self._inorder_values(node.left)
        yield node.value
        yield from self._inorder_values(node.right)

def items(self):
    """Return iterator over all key-value pairs in key order."""
    return self._inorder_items(self._root)

def _inorder_items(self, node):
    """Inorder traversal to get key-value pairs."""
    if node is not None:

```

```

        yield from self._inorder_items(node.left)
        yield (node.key, node.value)
        yield from self._inorder_items(node.right)

    def height(self):
        """Return height of the tree."""
        return self._subtree_height(self._root)

    def _subtree_height(self, node):
        """Calculate height of subtree."""
        if node is None:
            return 0
        return 1 + max(self._subtree_height(node.left),
                        self._subtree_height(node.right))

    def display_tree(self):
        """Display tree structure for visualization."""
        if self._root is None:
            print("Empty tree")
            return

    def _display_helper(node, level=0, prefix="Root: "):
        if node is not None:
            print(" " * (level * 4) + prefix + f"{node.key}: {node.value}")
            if node.left is not None or node.right is not None:
                _display_helper(node.left, level + 1, "L--- ")
                _display_helper(node.right, level + 1, "R--- ")

    _display_helper(self._root)

# Demonstrate BST sorted map
def demonstrate_bst_sorted_map():
    """Demonstrate binary search tree sorted map."""
    print("=== BST Sorted Map Demonstration ===")

    # Create sorted map
    sorted_map = BinarySearchTreeMap()

    # Add items in random order
    items = [(50, "Fifty"), (30, "Thirty"), (70, "Seventy"),
             (20, "Twenty"), (40, "Forty"), (60, "Sixty"), (80, "Eighty")]

    print("Adding items:")
    for key, value in items:
        sorted_map.put(key, value)
        print(f" Added {key}: {value}")

    print(f"\nTree structure:")
    sorted_map.display_tree()

    print(f"\nTree height: {sorted_map.height()}")
    print(f"Size: {sorted_map.size()}")

# Test sorted operations

```

```

print(f"\nSorted operations:")
print(f'First key: {sorted_map.first_key()}')
print(f'Last key: {sorted_map.last_key()}')
print(f'Before 45: {sorted_map.before(45)}')
print(f'After 45: {sorted_map.after(45)}')

# Range query
print(f'Keys in range [35, 65): {list(sorted_map.range(35, 65))}')

# Ordered iteration
print(f'All keys in order: {list(sorted_map.keys())}')
print(f'All items in order: {list(sorted_map.items())}')

demonstrate_bst_sorted_map()

```

AVL TREE FOR BALANCED SORTED MAP

To guarantee $O(\log n)$ performance, we need a balanced tree like AVL:

```

class AVLNode(BSTNode):
    """AVL tree node with height information."""

    def __init__(self, key, value, left=None, right=None, parent=None):
        super().__init__(key, value, left, right, parent)
        self.height = 1 # Height of the subtree rooted at this node

class AVLTreeMap(BinarySearchTreeMap):
    """Balanced sorted map using AVL tree."""

    def _get_height(self, node):
        """Get height of node (0 if None)."""
        return node.height if node is not None else 0

    def _update_height(self, node):
        """Update height of node based on children."""
        if node is not None:
            node.height = 1 + max(self._get_height(node.left),
                                 self._get_height(node.right))

    def _get_balance(self, node):
        """Get balance factor of node."""
        if node is None:
            return 0
        return self._get_height(node.left) - self._get_height(node.right)

    def _rotate_left(self, node):
        """Perform left rotation."""
        new_root = node.right
        node.right = new_root.left
        new_root.left = node

        # Update parent relationships
        new_root.parent = node.parent
        node.parent = new_root
        if node.right:

```

```

node.right.parent = node

    # Update heights
    self._update_height(node)
    self._update_height(new_root)

    return new_root

def _rotate_right(self, node):
    """Perform right rotation."""
    new_root = node.left
    node.left = new_root.right
    new_root.right = node

    # Update parent relationships
    new_root.parent = node.parent
    node.parent = new_root
    if node.left:
        node.left.parent = node

    # Update heights
    self._update_height(node)
    self._update_height(new_root)

    return new_root

def _rebalance(self, node):
    """Rebalance node if necessary."""
    if node is None:
        return node

    # Update height
    self._update_height(node)

    # Get balance factor
    balance = self._get_balance(node)

    # Left heavy
    if balance > 1:
        # Left-Right case
        if self._get_balance(node.left) < 0:
            node.left = self._rotate_left(node.left)
        # Left-Left case
        node = self._rotate_right(node)

    # Right heavy
    elif balance < -1:
        # Right-Left case
        if self._get_balance(node.right) > 0:
            node.right = self._rotate_right(node.right)
        # Right-Right case
        node = self._rotate_left(node)

    return node

```

```

def put(self, key, value):
    """Insert or update with automatic rebalancing. O(log n) time."""
    self._root = self._avl_insert(self._root, key, value, None)

    def _avl_insert(self, node, key, value, parent):
        """AVL insertion with rebalancing."""
        # Standard BST insertion
        if node is None:
            self._size += 1
            new_node = AVLNode(key, value, parent=parent)
            return new_node

        if key == node.key:
            node.value = value
        elif key < node.key:
            node.left = self._avl_insert(node.left, key, value, node)
        else:
            node.right = self._avl_insert(node.right, key, value, node)

        # Rebalance the tree
        return self._rebalance(node)

    def remove(self, key):
        """Remove with automatic rebalancing. O(log n) time."""
        if not self.contains(key):
            raise KeyError(f"Key '{key}' not found")

        value = self.get(key)
        self._root = self._avl_remove(self._root, key)
        self._size -= 1
        return value

    def _avl_remove(self, node, key):
        """AVL removal with rebalancing."""
        if node is None:
            return node

        if key < node.key:
            node.left = self._avl_remove(node.left, key)
        elif key > node.key:
            node.right = self._avl_remove(node.right, key)
        else:
            # Node to be deleted found
            if node.left is None:
                return node.right
            elif node.right is None:
                return node.left
            else:
                # Node with two children
                successor = self._subtree_min(node.right)
                node.key = successor.key
                node.value = successor.value
                node.right = self._avl_remove(node.right, successor.key)

```

```

    # Rebalance the tree
    return self._rebalance(node)

# Demonstrate AVL tree
def demonstrate_avl_tree():
    """Demonstrate AVL tree balanced sorted map."""
    print("=== AVL Tree Demonstration ===")

    # Create AVL tree
    avl_map = AVLTreeMap()

    # Add items that would create unbalanced BST
    keys = [10, 20, 30, 40, 50, 25] # Sequential insertion

    print("Adding items sequentially:")
    for key in keys:
        avl_map.put(key, f"Value_{key}")
        print(f" Added {key}, Height: {avl_map.height()}")

    print(f"\nFinal tree structure:")
    avl_map.display_tree()

    print(f"Tree height: {avl_map.height()}")
    print(f"Keys in order: {list(avl_map.keys())}")

demonstrate_avl_tree()

```

10.5 Skip Lists, Sets

SKIP LIST FUNDAMENTALS

A skip list is a probabilistic data structure that provides $O(\log n)$ expected time for search, insertion, and deletion operations. It uses multiple levels of linked lists with express lanes for faster traversal.

Skip List Structure:

```

Level 3: 1 -----> 7 -----> NIL
Level 2: 1 -----> 4 -> 7 -----> 9 -> NIL
Level 1: 1 -> 3 -> 4 -> 7 -> 8 -> 9 -> NIL
Level 0: 1 -> 3 -> 4 -> 7 -> 8 -> 9 -> NIL

```

Search Process: Start at highest level, move right until next key > target, then drop down one level.

```

import random

class SkipListNode:
    """Node for skip list implementation."""

    def __init__(self, key, value, level):
        self.key = key
        self.value = value
        self.forward = [None] * (level + 1) # Array of forward pointers

```

```

class SkipList(MapBase):
    """Skip list implementation of sorted map."""

    def __init__(self, max_level=16, p=0.5):
        """Initialize skip list.

        Args:
        max_level: Maximum number of levels
        p: Probability of promoting to next level
        """
        self.max_level = max_level
        self.p = p
        self._size = 0

        # Create header node with maximum level
        self.header = SkipListNode(None, None, max_level)
        self.level = 0 # Current level of skip list

    def _random_level(self):
        """Generate random level for new node."""
        level = 0
        while random.random() < self.p and level < self.max_level:
            level += 1
        return level

    def _search_path(self, key):
        """Find search path and return update array."""
        update = [None] * (self.max_level + 1)
        current = self.header

        # Start from highest level and move down
        for i in range(self.level, -1, -1):
            while (current.forward[i] is not None and
                  current.forward[i].key < key):
                current = current.forward[i]
            update[i] = current

        return update, current.forward[0]

    def put(self, key, value):
        """Insert or update key-value pair. O(log n) expected time."""
        update, current = self._search_path(key)

        # If key already exists, update value
        if current is not None and current.key == key:
            current.value = value
            return

        # Generate random level for new node
        new_level = self._random_level()

        # If new level is higher than current level, update header
        if new_level > self.level:
            for i in range(self.level + 1, new_level + 1):

```

```

        update[i] = self.header
self.level = new_level

    # Create new node
new_node = SkipListNode(key, value, new_level)

    # Update forward pointers
    for i in range(new_level + 1):
new_node.forward[i] = update[i].forward[i]
        update[i].forward[i] = new_node

self._size += 1

def get(self, key):
    """Retrieve value for key. O(log n) expected time."""
    _, current = self._search_path(key)

    if current is not None and current.key == key:
        return current.value
    raise KeyError(f"Key '{key}' not found")

def remove(self, key):
    """Remove key-value pair. O(log n) expected time."""
    update, current = self._search_path(key)

    if current is None or current.key != key:
        raise KeyError(f"Key '{key}' not found")

    value = current.value

    # Update forward pointers
    for i in range(self.level + 1):
        if update[i].forward[i] == current:
            update[i].forward[i] = current.forward[i]

    # Remove empty levels
    while self.level > 0 and self.header.forward[self.level] is None:
self.level -= 1

self._size -= 1
    return value

def contains(self, key):
    """Check if key exists. O(log n) expected time."""
    try:
self.get(key)
        return True
    except KeyError:
        return False

def is_empty(self):
    """Check if skip list is empty."""
    return self._size == 0

```

```

def size(self):
    """Return number of key-value pairs."""
    return self._size

def keys(self):
    """Return iterator over all keys in sorted order."""
    current = self.header.forward[0]
    while current is not None:
        yield current.key
        current = current.forward[0]

def values(self):
    """Return iterator over all values in key order."""
    current = self.header.forward[0]
    while current is not None:
        yield current.value
        current = current.forward[0]

def items(self):
    """Return iterator over all key-value pairs in key order."""
    current = self.header.forward[0]
    while current is not None:
        yield (current.key, current.value)
        current = current.forward[0]

def display_structure(self):
    """Display skip list structure for visualization."""
    print(f'Skip List (Level: {self.level}, Size: {self._size})')

    for level in range(self.level, -1, -1):
        print(f'Level {level}: ', end='')
        current = self.header.forward[level]
        while current is not None:
            print(f'{current.key} -> ', end='')
            current = current.forward[level]
        print("NIL")

# Demonstrate skip list
def demonstrate_skip_list():
    """Demonstrate skip list operations."""
    print("=== Skip List Demonstration ===")

    # Create skip list
    skip_list = SkipList(max_level=4, p=0.5)

    # Add items
    items = [(3, "Three"), (7, "Seven"), (1, "One"), (9, "Nine"),
             (4, "Four"), (8, "Eight")]

    print("Adding items:")
    for key, value in items:
        skip_list.put(key, value)
    print(f"Added {key}: {value}")

```

```

print(f"\nSkip list structure:")
skip_list.display_structure()

# Test operations
print(f"\nTesting operations:")
print(f'Get key 7: {skip_list.get(7)}')
print(f'Contains key 5: {skip_list.contains(5)}')
print(f'All keys: {list(skip_list.keys())}')

# Remove item
removed = skip_list.remove(4)
print(f'Removed key 4: {removed}')

print(f"\nAfter removal:")
skip_list.display_structure()

demonstrate_skip_list()

```

SET DATA STRUCTURE

A set is a collection of unique elements with no associated values. Sets can be implemented using the same underlying structures as maps.

```

class Set:
    """Set implementation using hash table."""

    def __init__(self):
        """Initialize empty set."""
        self._map = ChainHashMap()

    def add(self, element):
        """Add element to set. O(1) expected time."""
        self._map.put(element, True)

    def remove(self, element):
        """Remove element from set. O(1) expected time."""
        try:
            self._map.remove(element)
        except KeyError:
            raise KeyError(f'Element '{element}' not in set')

    def contains(self, element):
        """Check if element is in set. O(1) expected time."""
        return self._map.contains(element)

    def is_empty(self):
        """Check if set is empty."""
        return self._map.is_empty()

    def size(self):
        """Return number of elements."""
        return self._map.size()

    def union(self, other):
        """Return union of two sets."""
        result = Set()
        for element in self:

```

```

result.add(element)
    for element in other:
result.add(element)
    return result

def intersection(self, other):
    """Return intersection of two sets."""
    result = Set()
    for element in self:
        if element in other:
result.add(element)
    return result

def difference(self, other):
    """Return difference of two sets (self - other)."""
    result = Set()
    for element in self:
        if element not in other:
result.add(element)
    return result

def symmetric_difference(self, other):
    """Return symmetric difference of two sets."""
    return self.union(other).difference(self.intersection(other))

def is_subset(self, other):
    """Check if this set is subset of other."""
    for element in self:
        if element not in other:
            return False
    return True

def is_superset(self, other):
    """Check if this set is superset of other."""
    return other.is_subset(self)

def __iter__(self):
    """Make set iterable."""
    return self._map.keys()

def __len__(self):
    """Return size using len() function."""
    return self.size()

def __contains__(self, element):
    """Support 'in' operator."""
    return self.contains(element)

def __repr__(self):
    """String representation of set."""
    elements = list(self)
    return "{" + ", ".join(str(e) for e in elements) + "}"

# Demonstrate set operations

```

```

def demonstrate_set_operations():
    """Demonstrate set data structure and operations."""
    print("==== Set Operations Demonstration ====")

    # Create sets
    set1 = Set()
    set2 = Set()

    # Add elements to set1
    for element in [1, 2, 3, 4, 5]:
        set1.add(element)

    # Add elements to set2
    for element in [4, 5, 6, 7, 8]:
        set2.add(element)

    print(f"Set 1: {set1}")
    print(f"Set 2: {set2}")

    # Set operations
    print(f"\nSet Operations:")
    print(f"Union: {set1.union(set2)}")
    print(f"Intersection: {set1.intersection(set2)}")
    print(f"Set1 - Set2: {set1.difference(set2)}")
    print(f"Symmetric difference: {set1.symmetric_difference(set2)}")

    # Membership tests
    print(f"\nMembership tests:")
    print(f"3 in Set1: {3 in set1}")
    print(f"6 in Set1: {6 in set1}")

    # Subset tests
    subset = Set()
    subset.add(2)
    subset.add(3)
    print(f"Subset {subset}  $\subseteq$  Set1: {subset.is_subset(set1)}")

demonstrate_set_operations()

```

10.6 Multisets, and Multimaps

MULTISET IMPLEMENTATION

A multiset (or bag) allows duplicate elements and keeps track of element frequencies.

```

class Multiset:
    """Multiset implementation allowing duplicate elements."""

    def __init__(self):
        """Initialize empty multiset."""
    self._counts = ChainHashMap() # Maps elements to their counts
    self._size = 0

```

```

def add(self, element, count=1):
    """Add element(s) to multiset. O(1) expected time."""
    if count <= 0:
        return

    current_count = self._counts.get(element) if self._counts.contains(element) else 0
    self._counts.put(element, current_count + count)
    self._size += count

def remove(self, element, count=1):
    """Remove element(s) from multiset. O(1) expected time."""
    if not self._counts.contains(element):
        raise KeyError(f'Element '{element}' not in multiset")

    current_count = self._counts.get(element)
    if count >= current_count:
        # Remove all occurrences
        self._counts.remove(element)
    self._size -= current_count
    else:
        # Remove partial count
        self._counts.put(element, current_count - count)
    self._size -= count

def count(self, element):
    """Return count of element in multiset. O(1) expected time."""
    return self._counts.get(element) if self._counts.contains(element) else 0

def contains(self, element):
    """Check if element exists in multiset. O(1) expected time."""
    return self._counts.contains(element)

def is_empty(self):
    """Check if multiset is empty."""
    return self._size == 0

def size(self):
    """Return total number of elements (including duplicates)."""
    return self._size

def unique_elements(self):
    """Return number of unique elements."""
    return self._counts.size()

def elements(self):
    """Return iterator over all elements (with repetitions)."""
    for element in self._counts.keys():
        count = self._counts.get(element)
        for _ in range(count):
            yield element

def distinct_elements(self):
    """Return iterator over unique elements."""

```

```

    return self._counts.keys()

def most_common(self, n=None):
    """Return list of (element, count) pairs in descending count order."""
    items = [(element, self._counts.get(element))
              for element in self._counts.keys()]
    items.sort(key=lambda x: x[1], reverse=True)

    if n is not None:
        return items[:n]
    return items

def union(self, other):
    """Return union multiset (max counts)."""
    result = Multiset()

    # Add elements from self
    for element in self.distinct_elements():
        result.add(element, self.count(element))

    # Add elements from other (taking max count)
    for element in other.distinct_elements():
        current_count = result.count(element)
        other_count = other.count(element)
        if other_count > current_count:
            result.add(element, other_count - current_count)

    return result

def intersection(self, other):
    """Return intersection multiset (min counts)."""
    result = Multiset()

    for element in self.distinct_elements():
        if other.contains(element):
            min_count = min(self.count(element), other.count(element))
            result.add(element, min_count)

    return result

def __len__(self):
    """Return size using len() function."""
    return self.size()

def __contains__(self, element):
    """Support 'in' operator."""
    return self.contains(element)

def __iter__(self):
    """Make multiset iterable over all elements."""
    return self.elements()

def __repr__(self):
    """String representation of multiset."""

```

```

    if self.is_empty():
        return "Multiset()"

    items = []
    for element, count in self.most_common():
        if count == 1:
            items.append(str(element))
        else:
            items.append(f'{element} × {count}')

    return "Multiset({" + ", ".join(items) + "})"

# Demonstrate multiset operations
def demonstrate_multiset():
    """Demonstrate multiset data structure."""
    print("=== Multiset Demonstration ===")

    # Create multiset
    bag = Multiset()

    # Add elements with different frequencies
    elements = ["apple", "banana", "apple", "cherry", "banana", "apple"]
    print("Adding elements:")
    for element in elements:
        bag.add(element)
        print(f"Added '{element}', count now: {bag.count(element)}")

    print(f"\nMultiset: {bag}")
    print(f"Total size: {bag.size()}")
    print(f"Unique elements: {bag.unique_elements()}")

    # Most common elements
    print(f"Most common: {bag.most_common()}")

    # Create another multiset
    bag2 = Multiset()
    bag2.add("banana", 2)
    bag2.add("date", 1)
    bag2.add("apple", 1)

    print(f"\nSecond multiset: {bag2}")

    # Set operations
    print(f"Union: {bag.union(bag2)}")
    print(f"Intersection: {bag.intersection(bag2)}")

    # Remove elements
    bag.remove("apple", 2)
    print(f"\nAfter removing 2 apples: {bag}")

demonstrate_multiset()

```

MULTIMAP IMPLEMENTATION

A multimap allows multiple values to be associated with a single key.

```
class Multimap:
    """Multimap implementation allowing multiple values per key."""

    def __init__(self):
        """Initialize empty multimap."""
    self._data = ChainHashMap() # Maps keys to lists of values
    self._size = 0
```

Check Your Progress

Question 1: What is the primary advantage of hash tables over binary search trees? a) Hash tables maintain sorted order b) Hash tables provide $O(1)$ average-case operations c) Hash tables use less memory d) Hash tables handle collisions better

Question 2: In separate chaining, what happens when the load factor exceeds 1.0? a) The hash table stops working b) Performance degrades gradually but remains functional c) Automatic resizing occurs immediately d) Collisions become impossible

Question 3: Which probing strategy provides the most uniform distribution in open addressing? a) Linear probing b) Quadratic probing c) Double hashing d) All provide equal distribution

Question 4: In an AVL tree, what triggers a rebalancing operation? a) When tree height exceeds $\log n$ b) When balance factor of any node exceeds ± 1 c) When load factor exceeds 0.75 d) When insertion occurs

Question 5: Skip lists achieve $O(\log n)$ expected performance through: a) Deterministic balancing like AVL trees b) Hash-based indexing c) Probabilistic multilevel structure d) Binary search on sorted arrays

Question 6: Explain the trade-offs between separate chaining and open addressing for collision resolution.

Question 7: Describe the four rotation types in AVL trees and when each is applied.

Question 8: How does the probability parameter p affect skip list performance and structure?

Question 9: Compare the memory overhead of different map implementations.

Question 10: Explain why hash tables cannot provide ordered iteration while sorted maps can.

10.7 Summary

This unit explored the fundamental concepts of maps, dictionaries, and advanced data structures that enable efficient key-value storage and retrieval. The progression from simple list-based implementations to sophisticated probabilistic structures demonstrates the evolution of data structure design in computer science.

Core Concepts Discussed

- **Maps and Dictionaries:** We established the foundation with the map abstract data type, which stores unique key-value pairs and provides essential operations: put, get, remove, contains, and iteration methods. The simple list-based implementation revealed the $O(n)$ limitations that drive the need for more efficient structures.
- **Hash Tables:** The centerpiece of efficient map implementations, hash tables achieve $O(1)$ average-case performance through strategic hashing and collision resolution. We examined:
 - Hash function design principles (uniform distribution, determinism, efficiency)
 - Various hash functions (simple sum, polynomial rolling hash, DJB2)
 - Collision resolution strategies:
 - **Separate Chaining:** Uses linked lists at each bucket, maintaining performance even with high load factors
 - **Open Addressing:** Stores elements directly in the array using probing (linear, quadratic, double hashing)
- **Load Factor Management:** Critical for hash table performance, load factor ($\alpha = n/m$) determines when resizing is necessary. Performance degrades when α exceeds 0.7-0.8, requiring dynamic resizing and rehashing.
- **Sorted Maps:** Built on binary search trees, these structures maintain key ordering while providing $O(h)$ operations where h is tree height. Key features include:
 - Range queries and ordered iteration
 - Additional operations: first_key(), last_key(), before(), after()
 - AVL trees guarantee $O(\log n)$ performance through automatic balancing via rotations
- **Skip Lists:** Probabilistic alternatives to balanced trees that achieve $O(\log n)$ expected performance through multilevel "express lanes." Each node's level is determined randomly, creating a structure that's simpler to implement than self-balancing trees while maintaining comparable performance.
- **Extended Structures:**
 - **Sets:** Unique element collections implemented using map foundations
 - **Multisets:** Allow duplicate elements with frequency tracking
 - **Multimaps:** Support multiple values per key

Performance Analysis Summary

- The choice between implementations depends on specific requirements:
- Hash tables for fastest average-case operations without ordering
- Balanced trees for guaranteed performance with sorted access
- Skip lists for probabilistic guarantees with simpler implementation

Practical Applications

- These structures form the backbone of:
 - Database indexing systems
 - Compiler symbol tables
 - Web session management
 - Caching mechanisms
 - Configuration systems
 - Search engines and information retrieval

- **Question 1:** What is the primary advantage of hash tables over binary search trees? a) Hash tables maintain sorted order b) Hash tables provide $O(1)$ average-case operations c) Hash tables use less memory d) Hash tables handle collisions better
 - **Answer: b) Hash tables provide $O(1)$ average-case operations**
 - *Explanation:* Hash tables achieve constant-time average performance for basic operations, while BSTs require $O(\log n)$ for balanced trees or $O(n)$ for unbalanced trees.
-
- **Question 2:** In separate chaining, what happens when the load factor exceeds 1.0? a) The hash table stops working b) Performance degrades gradually but remains functional c) Automatic resizing occurs immediately d) Collisions become impossible
 - **Answer: b) Performance degrades gradually but remains functional**
 - *Explanation:* With separate chaining, load factors above 1.0 mean multiple elements per bucket on average, but the structure remains operational with longer chain searches.
-
- **Question 3:** Which probing strategy provides the most uniform distribution in open addressing? a) Linear probing b) Quadratic probing c) Double hashing d) All provide equal distribution
 - **Answer: c) Double hashing**
 - *Explanation:* Double hashing uses a second hash function to determine probe intervals, eliminating both primary and secondary clustering effects.
-
- **Question 4:** In an AVL tree, what triggers a rebalancing operation? a) When tree height exceeds $\log n$ b) When balance factor of any node exceeds ± 1 c) When load factor exceeds 0.75 d) When insertion occurs
 - **Answer: b) When balance factor of any node exceeds ± 1**
 - *Explanation:* AVL trees maintain the balance property where each node's left and right subtree heights differ by at most 1.
-
- **Question 5:** Skip lists achieve $O(\log n)$ expected performance through: a) Deterministic balancing like AVL trees b) Hash-based indexing c) Probabilistic multilevel structure d) Binary search on sorted arrays
 - **Answer: c) Probabilistic multilevel structure**
 - *Explanation:* Skip lists use random level assignment with probability p (typically 0.5) to create express lanes that enable logarithmic search time.
-
- Section B: Short Answer Questions**
- **Question 6:** Explain the trade-offs between separate chaining and open addressing for collision resolution.
 - **Answer: Separate Chaining:**
 - *Advantages:* Simple implementation, handles high load factors well, no clustering, easy deletion
 - *Disadvantages:* Extra memory for pointers, cache performance issues due to indirection
 - **Open Addressing:**
 - *Advantages:* Better cache locality, no extra memory for pointers, compact storage
 - *Disadvantages:* Performance degrades rapidly with high load factors, clustering issues, complex deletion (requires tombstones)

- **Choice depends on:** Expected load factor, memory constraints, and performance requirements.
-
- **Question 7:** Describe the four rotation types in AVL trees and when each is applied.
 - **Answer:**
 1. **Single Right Rotation (LL case):** Applied when left subtree is heavy and imbalance is in left child's left subtree
 2. **Single Left Rotation (RR case):** Applied when right subtree is heavy and imbalance is in right child's right subtree
 3. **Left-Right Rotation (LR case):** Applied when left subtree is heavy but imbalance is in left child's right subtree (requires left rotation on child, then right rotation on parent)
 4. **Right-Left Rotation (RL case):** Applied when right subtree is heavy but imbalance is in right child's left subtree (requires right rotation on child, then left rotation on parent)
-
- **Question 8:** How does the probability parameter p affect skip list performance and structure?
 - **Answer:** The probability p determines how many levels each node has:
 - **Lower p (e.g., 0.25):** Fewer high-level nodes, more compact structure, but potentially longer search paths
 - **Higher p (e.g., 0.75):** More high-level nodes, more express lanes, but higher memory overhead
 - **Standard $p = 0.5$:** Balances memory usage and performance, expected height $O(\log n)$
 - **Impact:** p affects the trade-off between space efficiency and search performance. $p = 0.5$ is optimal for most applications.
-
- **Question 9:** Compare the memory overhead of different map implementations.
 - **Answer:**
 1. **List-based map:** $O(n)$ for key-value pairs only
 2. **Hash table (chaining):** $O(n)$ for data + $O(m)$ for bucket array + pointer overhead
 3. **Hash table (probing):** $O(m)$ where $m \geq n$ for the array, no pointer overhead
 4. **BST/AVL tree:** $O(n)$ for data + $O(n)$ for node pointers (typically 2-3 pointers per node)
 5. **Skip list:** $O(n)$ for data + variable pointer overhead based on level distribution
 - **Most memory-efficient:** Open addressing hash table **Highest overhead:** Skip list (due to multilevel pointers)
-
- **Question 10:** Explain why hash tables cannot provide ordered iteration while sorted maps can.
 - **Answer: Hash Tables:** Use hash functions that distribute keys uniformly across buckets for performance, destroying any natural ordering. The hash value has no correlation with the key's sorted position.
 - **Sorted Maps:** Maintain keys in a structure that preserves ordering relationships (like BST in-order traversal), enabling operations like:
 - Ordered iteration
 - Range queries
 - Predecessor/successor finding

- **Trade-off:** Hash tables sacrifice ordering for $O(1)$ average performance, while sorted maps maintain ordering at the cost of $O(\log n)$ operations.

Linked List

10.9 Further Readings

- https://en.wikipedia.org/wiki/Skip_list



UNIT 12 SORTING ALGORITHMS

Structure

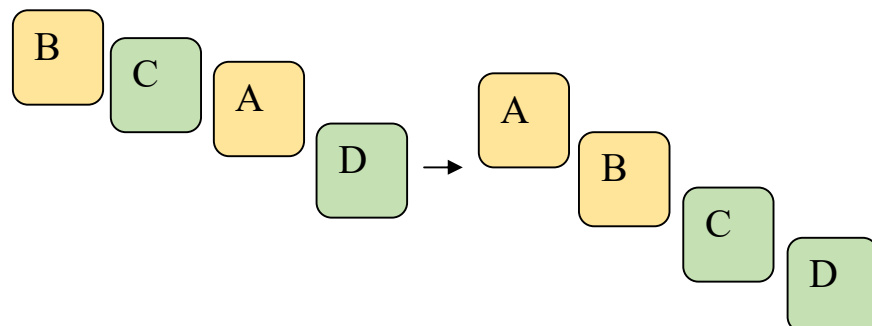
- 12.0 Introduction
- 12.1 Objectives
- 12.2 Why Study Algorithms?
- 12.3 Categories of Sorting Algorithms
- 12.4 Bubble Sort
- 12.5 Selection Sort
- 12.6 Insertion Sort
- 12.7 Merge Sort
- 12.8 Quick Sort
- 12.9 Comparing Various Sorting Algorithms
- 12.10 Python's Built-In Sorting Function
- 12.11 Summay
- 12.12 Solutions/Answers to Check Your Progress
- 12.13 Further Readings

12.0 Introduction

Sorting algorithms are directly related to databases as there is a need to sort records available in the database. Suppose, the database consists of details of students, then there is a need for names in alphabetical order, roll no in ascending/ descending order, etc. Sorting is a preliminary step involved while dealing with databases and thus is an important area to study in computer science as every project deals with a dataset. Certain algorithms have been developed for sorting the data, and this area is still an area of research.

In this chapter, we will discuss sorting algorithms, namely, merge sort, selection sort, bubble sort, etc. Each of the algorithms is demonstrated using examples and python codes. Let's study sorting algorithms one by one.

"Sorting is the method of arranging elements in either ascending or descending order."



12.1 Objectives

After going through this unit, you should be able to

- Understand the concept of sorting;
- Remember various terminologies related to sorting
- Articulate various sorting algorithms
- Implement python codes for performing sorting

12.2 Why Study Sorting Algorithms

In the field of Computer Science, everything is related to databases. When we talk about a database that contains various columns, sorting algorithms are needed to arrange the data in a particular order (increasing/ decreasing). Sorting is an important category of algorithms in the field of Computer Science and a lot of research is still going on.

12.3 Categories of Sorting Algorithms

Sorting algorithms are generally compared on the following parameters:

1. *Number of Comparisons*

The most important parameter to judge a sorting algorithm is the number of comparisons done to sort a given list of items. In this chapter, we will discuss both comparison sorting algorithms.

The comparison sorting algorithms consist of merge sort, quick sort, bubble, insertion, and selection sort.

Whereas, non-comparison sorting algorithms consist of count sort, bucket sort, and radix sort.

2. *Memory Usage*

The sorting algorithms are usually divided into two types, i.e., in-place and not in-place algorithm.

Sorting algorithm that do not need additional memory for sorting process is known as in-place algorithms.

3. *Stability*

The algorithm that preserves the relative order of same elements during sorting is known as **stable sorting algorithm**. That means, if two items have same value, the item that appeared first in the unsorted list should also appear first in the sorted list.

4. *External vs Internal Sort*

Algorithms that use main memory during sorting are called **internal sort**.

Algorithms that use external memory devices such as disk or tape during sorting are known as **external sort**.

Let's discuss various sorting algorithms as listed below one by one:

- a. Bubble Sort
- b. Selection Sort
- c. Insertion Sort
- d. Merge Sort
- e. Quick Sort

You will learn about the category of each algorithm i.e., whether an algorithm is in-place or not, stable or not, during detailed discussion of particular sorting algorithm.

Check your Progress – I

- a. What is the difference between external and internal sorting algorithms?
- b. What do you mean by term stable algorithm?
- c. Define the term in-place algorithm.

12.4 Bubble Sort

Bubble sort is the simplest algorithm that works by comparing each pair of elements and swapping elements if they are in the wrong order. The algorithm continues iteration until no more swaps are required. Bubble sort is not suitable for large datasets due to its high time complexity.

Bubble sort does not require additional memory space, i.e., in-place and is stable sorting algorithm.

Working of Bubble Sort

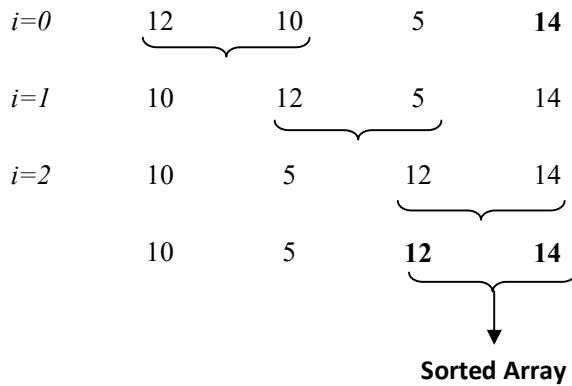
1. The array is sorted using multiple passes.
2. The largest element goes to the end of array after first pass. The second largest element goes to second last position after second pass and so on.
3. In all passes, we compare all adjacent elements and swap of larger element is before the smaller element.

Let's understand this with help of an example:

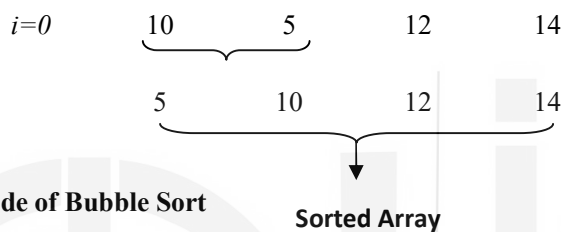
First Pass:



Second Pass:



Third Pass:



Pseudocode of Bubble Sort

```
def Bubblesort (A)
    for i in range (len(A)):
        for j in range (len(A)-1, i, -1):
            if (A[j] < A[j-1]):
                swap (A, j, j-1)
```

Performance Analysis:

	Time Complexity	Reason
Worst Case	$O(n^2)$	When array is arranged in descending order
Average Case	$O(n^2)$	Irrespective of order of elements
Best Case	$O(n^2)$	Already sorted array

Space Complexity: Bubble sort does not require additional memory space i.e., space complexity is $O(1)$.

Check Your Progress-II

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using bubble sort.
- Sort the given array of numbers using bubble sort.
67, 21, 12, 10, 9, 7, 5
- Is bubble sort a stable algorithm? Why/ Why not?

12.5 Selection Sort

Selection sort works by repeatedly selecting the smallest element from the unsorted

array and swapping it with the first unsorted element. The process continues till the array is sorted. The algorithm is easy to understand and does not require additional memory space for sorting. But it does not preserve the relative order of elements with the same value i.e., unstable sorting algorithm.

Working of Selection Sort

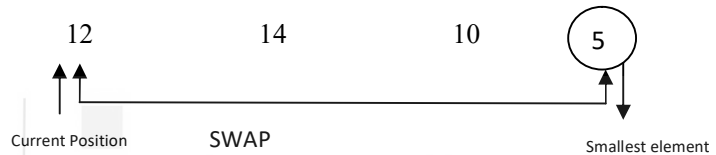
1. Find the minimum element in the unsorted array and swap it with element at current position.
2. Repeat this process until all elements are in correct order i.e., the array is sorted.

Let's understand this with the help of an example:

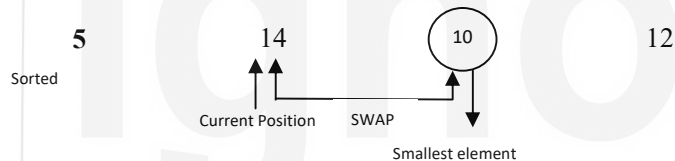
Considering the given array as:

12 14 10 5

Step 01



Step 02



Step 03



Sorted Array

5 10 12 14

Pseudocode of Selection Sort

```
def Selectionsort (A):
```

```
    for i in range (len (A)):
```

```
        min = i
```

```
        for j in range (i+1, len (A)):
```

```
            if (A[j] < A[min])
```

```
                min = j
```

```
        swap (A, min, i)
```

Performance Analysis

	Time Complexity	Reason
Worst Case	$O(n^2)$	The time complexity is irrespective of order of the elements
Average Case	$O(n^2)$	
Best Case	$O(n^2)$	

Space Complexity: It is an in-place sorting algorithm i.e., does not require additional memory space i.e., space complexity is $O(1)$.

Check Your Progress-III

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using selection sort.
- Show all steps to sort given array 90, 45, 11, 5, 67, 43 using selection sort.

12.6 INSERTION SORT

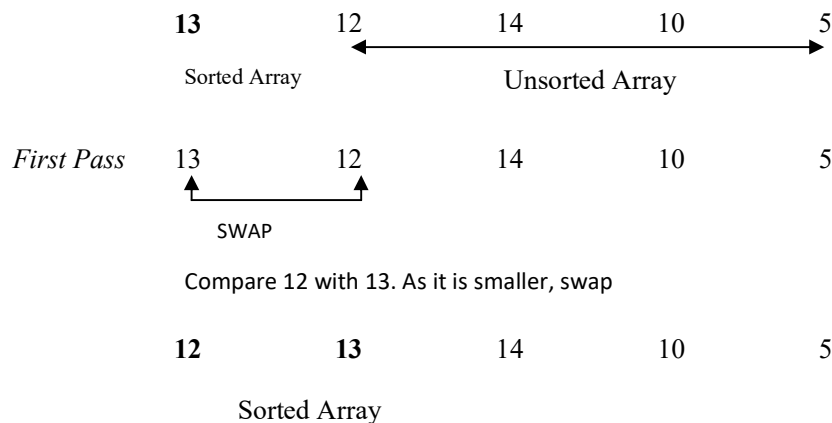
It is a simple sorting algorithm that divide the array into two parts i.e., sorted and unsorted. The algorithm works by iteratively inserting each element of unsorted list into the right place in the sorted list.

Insertion sort is efficient for small data and is a stable and in-place algorithm.

Working of Insertion Sort:

- The first element of the array is assured to be sorted.
- Compare the second element with the element in the sorted array if it is smaller than swap.
- Compare the third element with elements in the sorted array and put it in the correct position.
- Repeat the above steps with other elements until the array is sorted.

Let's understand this with the help of an example:



Second Pass 12 13 14 10 5

Compare 14 with elements in sorted array & swap where required

12 13 14 10 5

Third Pass 12 13 14 10 5

Compare 10 with elements in sorted array & swap where required

12 13 10 14 5

Swap

12 10 13 14 5

Fourth Pass

10 12 13 14 5

10 12 13 14 5

10 12 13 5 14

10 12 5 13 14

10 5 12 13 14

5 10 12 13 14

Pseudocode

def Insertionsort (A):

 for i in range (1, len (A)):

 temp = A[i]

 j = i

 while (j >0 and tmp< A[j-1]):

 A[j] = A[j-1]

 j = j -1

 A[j] = temp

Performance Analysis:

	Time Complexity	Reason
Worst Case	$O(n^2)$	When array is arranged in descending order
Average Case	$O(n^2)$	Irrespective of order of elements
Best Case	$O(n)$	Already sorted array

Space Complexity:

It does not require any auxiliary space i.e., $O(1)$ space complexity. Insertion sort is usually preferred when the array is nearly sorted or when the input size is small.

Check Your Progress-IV

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using insertion sort.

12.7 Merge Sort

Merge sort is popular sorting algorithm that works on Divide and Conquer strategy. It works by recursively dividing i/p array into two halves, sorting both halves and merging them together to obtain sorted array. It is a stable and not in-place algorithm.

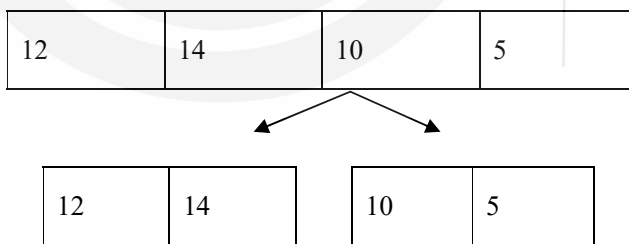
Working of Merge Sort

1. Divide the unsorted array into parts until it cannot be sub-divided- Divide
2. Conquer each sub-array created and sort them – Conquer
Note – If an array contains single element, it is considered sorted.
3. Merge the sorted sub-arrays into correct order.

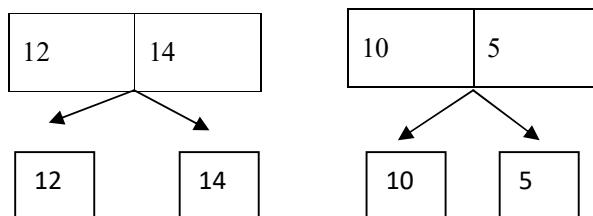
Let's understand it with help of an example:

Given Array: 12 14 10 5

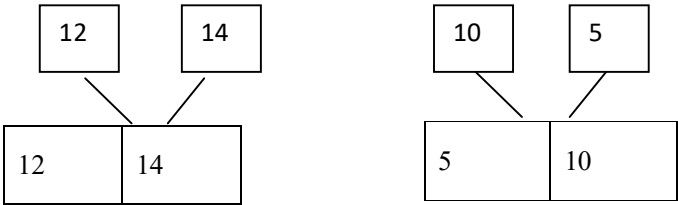
Pass 01: Split the array into equal halves



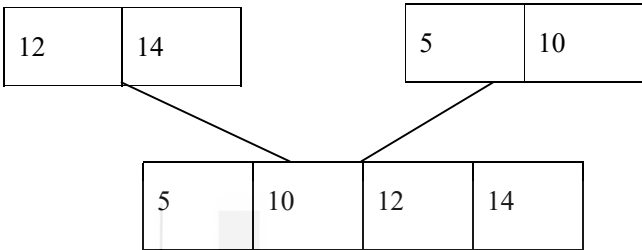
Pass 02: Split the sub-arrays further into equal halves



Pass 03: Sub-arrays cannot be divided further. Therefore, we move to step 02 i.e., conquer, sort, and merge



Pass 04: Conquer, sort , and merge



Pseudocode of Merge Sort:

```
def mergesort (X):
    mid = len (X)/2
    leftpart = X [:mid]
    rightpart = X[mid:]
    mergesort (leftpart)
    mergesort (rightpart)
    while i<len (leftpart) and j <len (rightpart):
        if leftpart[i] <rightpart[j]
            X[k] = leftpart[i]
            i = i + 1
        else
            X[k] = rightpart [j]
            j +=1
        k = k + 1
    while i<len (leftpart):
        X[k] = leftpart [j]
        j = j + 1
        k = k + 1
```

Performance Analysis:

Time Complexity		Reason
Worst Case	$O(n \log n)$	When array is arranged in descending order

Average Case	$O(n \log n)$	Irrespective of order of elements
Best Case	$O(n \log n)$	Already sorted array

Space Complexity:

It is not an in-place sorting algorithm i.e., needs additional memory space of $O(n)$.

Check Your Progress V

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using bubble sort.

12.8 Quick Sort

Quick sort works on divide and conquer strategy and is also known as partition exchange sort. Just like merge sort, it uses recursive calls to sort the array. The algorithm picks an element as a pivot and partitions the unsorted array around pivot with an aim of placing pivot in the correct position in the sorted array. Further, quick sort is not a stable algorithm i.e., it does not preserve the relative order of same elements. It is in-place algorithm that do not require auxiliary space for sorting.

Working of Quick Sort

- Choose an element as pivot (leftmost element is usually taken as pivot).
- Split the array into 2 parts such that elements smaller than pivot are placed in one part and elements greater than pivot are placed in another part.
- Repeat the same process on both sub-arrays.

Let's understand this with help of an example:

12 14 10 5

Step 01: Considering last element i.e. 5 as pivot

All values are greater than 5 and therefore should be to right of pivot. Swap 5 and 12

5 14 10 12

Step 02: Consider 12 as pivot

Value 10 should be on left of pivot

Value 14 should be on right of pivot

Swap 10 and 14

5 10 14 12

Again, swap 12 and 14

5 10 12 14

Pseudocode of Quicksort

def Partition (X, low, high):

pivot = low

```

swap ( X, pivot, high)
for i in range ( low, high)
    if X[i] <= X[high]
        swap (X, I, low)
        low +=1

swap (X, low, high)
def Quicksort (X, low, high)
    if low < high:
        pivot = Partition (X, low, high)
        Quicksort (X, low, pivot -1 )
        Quicksort (X, pivot +1, high)

```

Performance Analysis

	Time Complexity	Reason
Worst Case	$O(n^2)$	When pivot is the largest or smallest value
Average Case	$O(n \log n)$	When partitions are balanced as much as possible
Best Case	$O(n \log n)$	When pivot divides array roughly into two equal halves

Space Complexity

It is an in-place algorithm, i.e., does not take any auxiliary space.

12.9 Comparing Various Sorting Algorithms

In this section, we will discuss best, average, and worst case time complexity of various sorting algorithms.

Refer to Table 1 for all information about space and time complexity of algorithms.

Table 2 Comparative Analysis of Various Sorting Algorithms

Algorithm	Worst Case	Average Case	Best Case	Stable	In-place
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	No
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	No	Yes

12.10 Python's Built-In Sorting Functions

There are two main built-in functions in Python for sorting list i.e. `list.sort()` and `sorted`.

Let's understand this with help of an example:

1. `List.sort()`

Given a list of numbers, it sorts the list in ascending or descending order without using any additional space. It makes changes in the original list and does not return a new list.

For instance-

```
l1 = [2, 3, 1, 4]
```

```
l1.sort()
```

It sorts the given list into ascending order

```
l1.sort(reverse = True)
```

It sorts list into descending order.

2. `Sorted()`

This method returns the new sorted list keeping the original list unchanged.

```
L1 = [2, 4, 1, 5]
```

```
L2 = sorted (L1)
```

```
L3 = sorted (L1, reverse = True)
```

Using `sorted()`, we can sort list, tuple or string.

12.11 Summary

- Sorting is the method of arranging elements in either ascending or descending order.
- The algorithm that preserves the relative order of the same elements during sorting is known as a stable sorting algorithm.
- The algorithm that does not need extra space for sorting is known as in-place sorting algorithm.
- Bubble sort works by comparing each pair of elements and swapping elements if they are in the wrong order.
- Selection sort works by repeatedly selecting the smallest element from the unsorted array and swapping it with the first unsorted element.
- Insertion sort is a simple sorting algorithm that divides the array into two parts.
- Merge sort is a popular sorting algorithm that works on Divide and Conquer strategy.

- Quick sort works on divide and conquer strategy and also known as partition exchange sort.

12.12 Solutions/ Answers to Check Your Progress

Check Your Progress I

1.

Internal Sorting	External Sorting
Sorting is done in the main memory (RAM)	Sorting is performed using external memory such as hard disk or secondary storage
Generally faster due to high memory access speed.	Generally slower due to frequent disk I/O operations.
Examples: Quick Sort, Merge Sort, Heap Sort, Bubble Sort	Examples: External Merge Sort, Polyphase Merge Sort

2. A stable sorting algorithm is a method that maintains the relative order of items with equal values.

Example: If a student has equal marks in a subject A and subject B & marks appear in the list as A before B, after stable sorting marks will still be written as A before B.

Bubble Sort, Merge Sort, and Insertion Sort are examples of stable algorithms.

Quick Sort, Heap Sort, and Selection Sort are examples of unstable algorithms.

3. An in-place algorithm is a sorting algorithm that do not require extra amount of memory for sorting elements i.e., can perform sorting in constant time.

It rearranges elements within the original structure and does not use additional data structures like extra arrays or lists; instead,

Quick Sort, Selection Sort, Bubble Sort, and Heap Sort are examples of in-place algorithms.

Check Your Progress-II

a. Given an array: 23, 90, 21, 54, 76

First Pass:

i=0	23	90	21	54	76
i=1	23	90	21	54	76
i=2	23	21	90	54	76
i=3	23	21	54	90	76

Second Pass:

Third Pass:

Fourth Pass:

b. 67, 21, 12, 10, 9, 7, 5

First Pass:

85

21	12	10	9	7	67	5
21	12	10	7	9	5	67

↓

Largest Element in Last (Sorted Array)

Second Pass:

i=0	21	12	10	7	9	5	67
i=1	12	21	10	7	9	5	67
i=2	12	10	21	7	9	5	67
	12	10	7	21	9	5	67
	12	10	7	9	21	5	67
	12	10	7	9	5	21	67

↓

Sorted Array

Third Pass:

i=0	12	10	7	9	5	21	67
i=1	10	12	7	9	5	21	67
i=2	10	7	12	9	5	21	67
i=3	10	7	9	12	5	21	67
i=4	10	7	9	5	12	21	67

Sorted Array

Fourth Pass:

i=0	10	7	9	5	12	21	67
i=1	7	10	9	5	12	21	67
i=2	7	9	10	5	12	21	67
	7	9	5	10	12	21	67

Sorted Array

Fifth Pass:

i=0	7	9	5	10	12	21	67
	}						
i=1	7	9	5	10	12	21	67
		}					
i=2	7	5	9	10	12	21	67
			}				

Sixth Pass:

i=0	7	5	9	10	12	21	67
	}						
	5	7	9	10	12	21	67

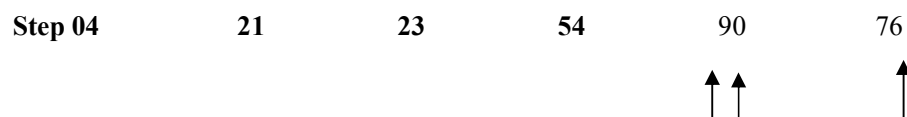
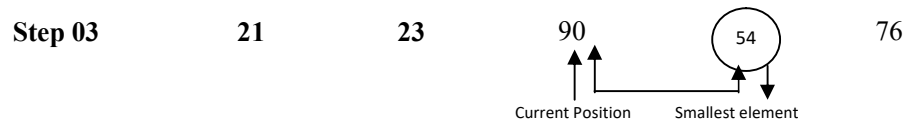
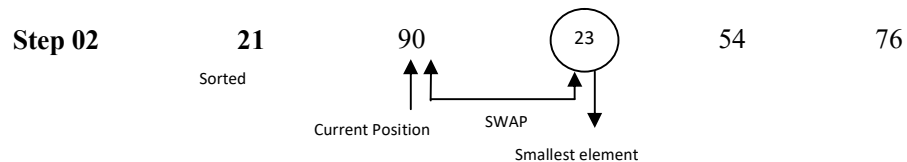
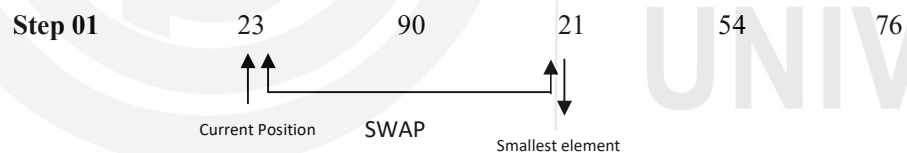
Sorted Array

5 7 9 10 12 21 67

- c. Bubble sort is a stable algorithm as it do not change the position of elements having the same value.

Check Your Progress III

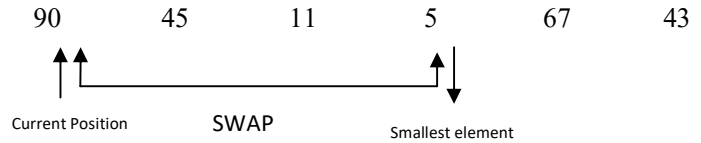
- a. Given 23, 90, 21, 54, 76



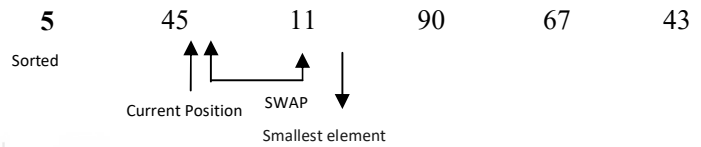
Sorted Array 21 23 54 76 90

b. Given array 90, 45, 11, 5, 67, 43

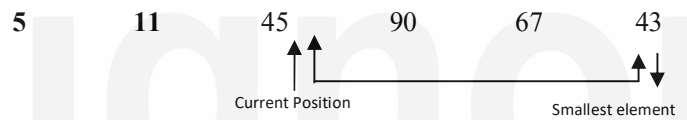
Step 01



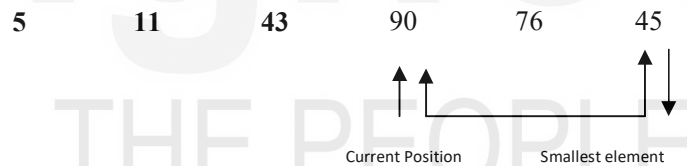
Step 02



Step 03



Step 04



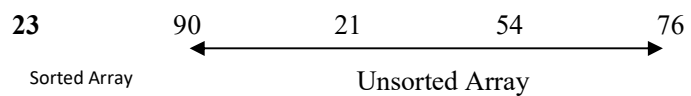
Step 05



Sorted Array 5 11 43 45 76 90

Check Your Progress IV

a. Given the array: 23, 90, 21, 54, 76



First Pass 23 90 21 54 76

 ↑ ↑

Compare 23 with 90. As 90 is greater, no swap

23 90 21 54 76

Sorted Array

Second Pass 23 90 21 54 76

 └───┘

Compare 21 with elements in sorted array & swap where required

23 21 90 54 76

└───┘

21 23 90 54 76

Third Pass 21 23 90 54 76

 └───┘

Compare 54 with elements in sorted array & swap where required

21 23 54 90 76

└───┘

No Swap

21 23 54 90 76

Fourth Pass 21 23 54 90 76

 └───┘

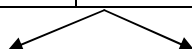
Sorted Array 5 10 12 13 14

Check Your Progress V

Given the array 90, 23, 54, 21

Pass 01: Split the array into equal halves

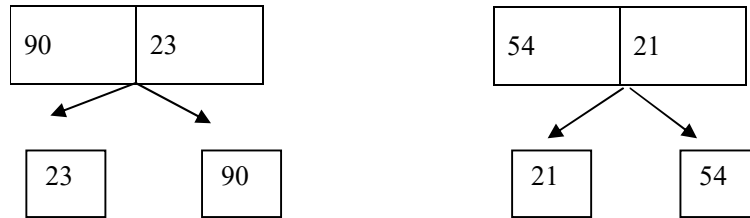
90	23	54	21
----	----	----	----



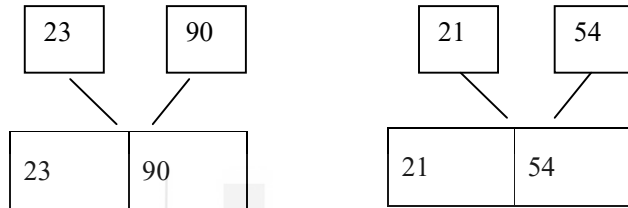
90	23
----	----

54	21
----	----

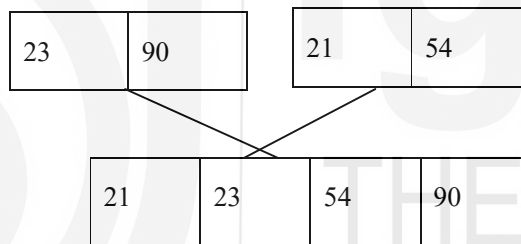
Pass 02: Split the sub-arrays further into equal halves



Pass 03: Sub-arrays cannot be divided further. Therefore, we move to step 02 i.e., conquer, sort, and merge



Pass 04: Conquer, sort, and merge



12.13 Further Readings

Reference Books

1. *Data Structure and Algorithmic Thinking with Python* by Narasimha Karumanchi, CareerMonk Publications, 2016.
2. *Data Structures & Algorithms in Python* by John Canning, Alan Broder, Robert Lafore, Addison-Wesley Professional, 1st edition, 14 October 2022.

Reference Websites

1. https://onlinecourses.nptel.ac.in/noc25_cs59/preview
2. https://www.udemy.com/course/data-structures-algorithms-in-python/?srsltid=AfmBOooSCmtk3ZRIIhQowVXciAk_Ta0HrHO9EPPzVSuM9WR8lvUqH2hZ&couponCode=CP2511241BG1